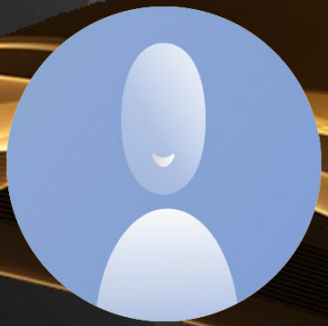
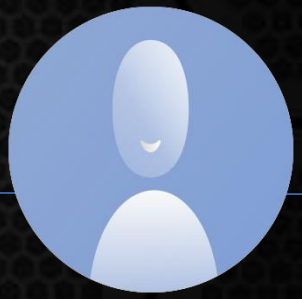
U7 – Arboles





Arboles



Trees

Arboles (Trees)

U5

CORE GENERAL ARBOLES

1 UNIDAD 6
6.1 Introducción

2
6.2 Árboles binarios
6.2.1 Operaciones en árboles binarios
6.2.1 Operaciones en árboles binarios
6.2.2 Árboles binarios de búsqueda

3
6.3 Árboles balanceados
6.3.1 Reacomodo del árbol
6.3.2 Inserción en árboles balanceados
6.3.3 Eliminación en árboles balanceados

6.4 Árboles-B
6.4.1 Búsqueda en árboles-B
6.4.2 Inserción en árboles-B
6.4.3 Eliminación en árboles-B

6.5 Árboles-B+
6.5.1 Búsqueda en árboles-B+
6.5.2 Inserción en árboles-B+
6.5.3 Eliminación en árboles-B+





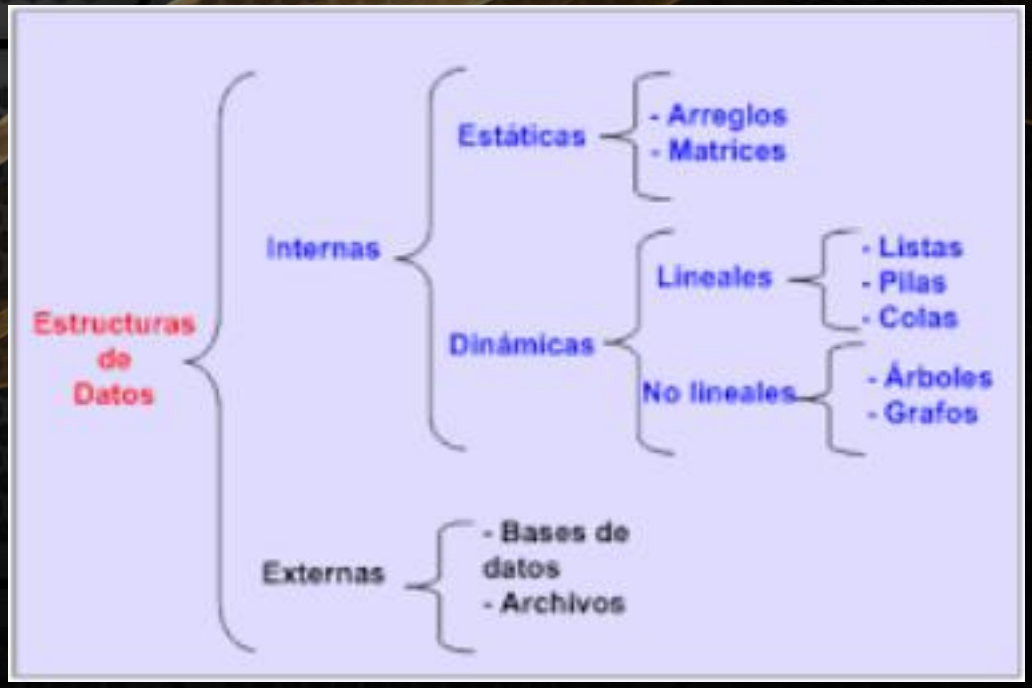
| Estructuras | - Repaso Introducción



Revisemos primero la clasificación de las estructuras de datos como lineales y no lineales.

Se dice que una estructura de datos es lineal si sus elementos forman una secuencia o una lista lineal. En una estructura de datos lineal, cada elemento de datos tiene un sucesor único y un predecesor único. Hay dos formas básicas de representar estructuras lineales en la memoria. Una forma es tener la relación entre los elementos por medio de punteros (links), llamados como listas enlazadas. Otra forma es usar la organización secuencial, es decir, los Array.

Las estructuras de datos no lineales se utilizan para representar los datos que contienen una relación jerárquica o de red entre los elementos. Los árboles y los gráficos son ejemplos de estructuras de datos no lineales. En las estructuras de datos no lineales, cada elemento de datos puede tener más de un predecesor y sucesor. Los elementos no forman ninguna secuencia lineal en particular.





Grafos | BASIC TERMINOLOGY

Introducción.

2.1

Graph

Grafos | Basic Terminology

Primero deberíamos aprender acerca de un **gráfico general** porque los árboles pueden verse como gráficos restringidos.

Un gráfico G consta de un conjunto no vacío V, un conjunto E y un mapeo del conjunto E al conjunto V. Aquí, V es el conjunto de nodos, también llamados puntos de vértices, del gráfico, y E es el conjunto de aristas del gráfico. Para gráficos finitos, V y E son finitos. Podemos representarlos como G = (V, E).



Estaremos Conceptualizando lo Siguiente:

Nodos adyacentes
Gráficos dirigidos y no dirigidos Bordes paralelos y multigrafico Gráfico ponderado
Gráfico nulo y vértice aislado
Grado de vértice
Caminos y circuitos
Conectividad
Árboles Bosque y arboles

Adjacent Nodes
Directed and Undirected Graphs
Parallel Edges and Multigraph
Weighted Graph
Null Graph and Isolated Vertex
Degree of Vertex
Paths and Circuits
Connectivity
Trees
Forest and Trees

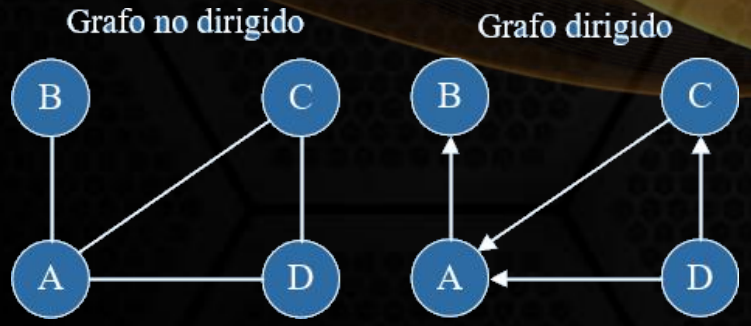


Grafos | Basic Terminology



Gráficos dirigidos y no dirigidos

En un gráfico G (V, E), un borde que se dirige de un nodo a otro se llama borde dirigido, mientras que un borde que no tiene una dirección específica se llama borde no dirigido. Un gráfico en el que se dirige cada borde se denomina gráfico o diagrama dirigido. Un gráfico en el que cada borde no está dirigido se denomina gráfico no dirigido. Si algunos de los bordes están dirigidos y algunos no están dirigidos en un gráfico, el gráfico se denomina gráfico mixto. Un mapa de la ciudad que muestra solo las calles de un solo sentido es un ejemplo de un gráfico dirigido donde las intersecciones son vértices y los bordes son calles. Un mapa que muestra solo las calles de dos sentidos es un ejemplo de un gráfico no dirigido, y un mapa que muestra todas las calles de un solo sentido y de dos sentidos es un ejemplo de un gráfico mixto.
Los arcos de un grafo dirigido se representan gráficamente con flechas. Un grafo no dirigido es un grafo donde los arcos conectan a los nodos en ambos sentidos.



Nodos adyacentes

Si un borde e Œ E está asociado con un par de nodos (a, b) donde a, b Œ V, entonces se dice que el borde e une o conecta los nodos ay b. Dos
nodos cualesquiera que estén conectados con un borde se denominan nodos adyacentes.

Bordes paralelos y multigrafico

El gráfico de la figura 7.2 (a) tiene solo un borde entre cualquier par de nodos. En los bordes ed directos, los dos posibles bordes entre el par de nodos que son opuestos en dirección se consideran distintos. En algunos gráficos tanto dirigidos como no dirigidos, puede haber más de un borde incidente en el mismo par de nodos, digamos ay b. En la figura 7.2 (b), las aristas e1, e2 y e3 son incidentes a los vértices ay b. Estos bordes se denominan bordes paralelos. Aquí, e1, e2 y e3 son tres aristas paralelas. Además, e5 y e6 son dos aristas paralelas. Cualquier gráfico que contenga aristas paralelas se denomina multigrafo. Por otro lado, una gráfica que no tiene bordes paralelos se llama gráfica simple.

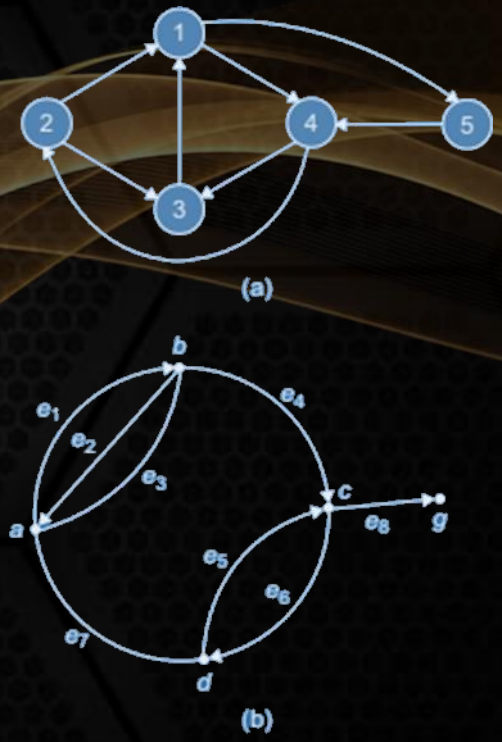


Fig. 7.2 Classification based on edges (a) Simple graph (b) Multigraph

Grafos | Basic Terminology

Gráfico ponderado

Un gráfico en el que se asignan pesos a cada borde se denomina gráfico ponderado. También se pueden asignar pesos a los vértices. A un gráfico de áreas y calles de una ciudad se le pueden asignar pesos de acuerdo con su densidad de tráfico. A un gráfico de áreas y carreteras de conexión se les pueden asignar pesos de manera que la distancia entre las ciudades se asigne a los bordes y la población del área se asigne a los vértices.

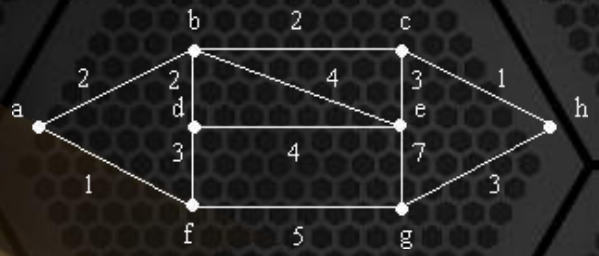


Gráfico nulo y vértice aislado

En un gráfico, un nodo que no es adyacente a ningún otro nodo se denomina nodo aislado. Un gráfico que contiene solo nodos aislados se denomina gráfico nulo. Por tanto, el conjunto de aristas es un conjunto vacío en un gráfico nulo. Sea V = conjunto de estudiantes, E = {existe un incidente de borde para dos estudiantes si comparten libros}. Sea V = {a, b, c}. Si no hay dos estudiantes entre a, byc que compartan libros, entonces la gráfica G = {V, E} se representa como se muestra en la figura 7.3 (a).

Aquí, G es un gráfico nulo y a, byc son vértices aislados. En la figura 7.3 (b), V = {a, b, c, d, e} y E = {(a, b), (a, c), (b, c), (c, d), (b, d)}, ye es un vértice aislado

Grado de vértice

El grado de un vértice en un grafo es el número de aristas incidentes a él. Un vértice aislado es un vértice con grado cero; esto es, un vértice que no es punto final de ninguna arista. Un vértice hoja es un vértice con grado uno. En un grafo dirigido, se puede distinguir entre grado de salida ("outdegree", número de aristas que salen del vértice) y grado de entrada ("indegree", número de aristas que llegan al vértice); un vértice fuente es un vértice con grado de entrada cero, mientras que un sumidero es un vértice con grado de salida cero.). En un gráfico no dirigido, el grado total o el grado de un nodo es el número de bordes incidentes al nodo. El grado del vértice aislado es cero. El grado del vértice a en la figura 7.4 es 3, mientras que el grado del vértice f es 1. Para el vértice 1 en la figura 7.2 (a), el grado entrante es 2 y el grado saliente es 2

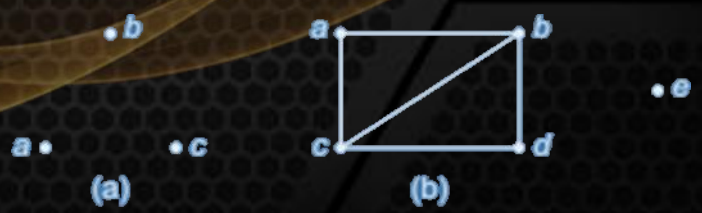


Fig. 7.3 Classification based on nodes (a) Null graph (b) Graph with isolated vertex



Fig. 7.4 Graph G



Grafos | Basic Terminology



Rutas y circuitos

Sea G = (V, E) una gráfica simple. Considere una secuencia de bordes de G tal que el nodo terminal de cualquier borde en la secuencia sea el nodo inicial del siguiente borde, si lo hay, en la secuencia (figura 7.4). Aquí, G = (V, E), V = {a, b, c, d} y E = {e1, e2, e3, e4, e5, e6, e7}. Un ejemplo de tal secuencia viene dado por {e1, e2, e4, e5}. La secuencia {e1, e2, e4, e5} también se puede escribir como {a, b, c, d, b}. Además, {e6, e2, e1, e3, e4, e2, e5} es otra secuencia. Tenga en cuenta que no todos los bordes y nodos que aparecen en una secuencia deben ser distintos. Además, para un gráfico dado, cualquier conjunto arbitrario de nodos como {a, f, b} que esté escrito en cualquier orden no da una secuencia como se requiere. De hecho, cada nodo que aparece en la secuencia debe ser adyacente a los nodos que aparecen justo antes y después de él en la secuencia, excepto el primero y el último nodos.



Fig. 7.4 Graph G

Conectividad

Se dice que un gráfico está conectado si y solo si existe una ruta entre cada par de vértices. En las figuras 7.6 (a) y (b) se muestran algunos ejemplos. La gráfica G = (V, E) dibujada en la figura 7.6 (a) con V = {a, b, c, d, e} es una gráfica desconectada. Contiene dos componentes conectados. Un gráfico conectado tiene un solo componente conectado. El gráfico que se muestra en la figura 7.6 (b) es un gráfico conectado.

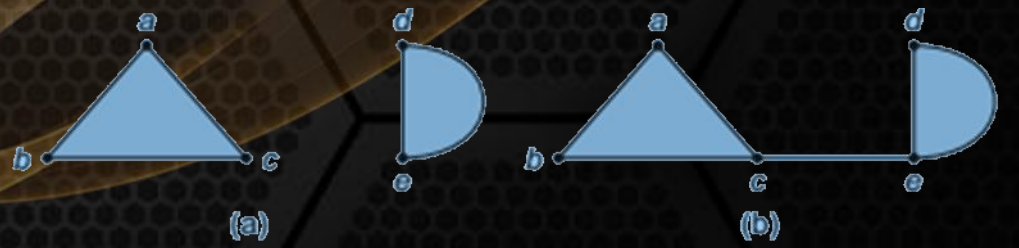


Fig. 7.6 Graph connectivity (a) Graph with two connected components (b) Connected graph with one connected component

Grafos + Arboles | Basic Terminology



Gráfico acíclico

Un gráfico simple que no tiene ciclos se llama gráfico acíclico. Estos gráficos no tienen bucles.

Árboles

Una clase de gráficos acíclicos se denomina árboles. Analicemos ahora una clase importante de gráficos llamados árboles y su terminología asociada. Los árboles son útiles para describir cualquier estructura que involucre jerarquía. Ejemplos familiares de tales estructuras son los árboles genealógicos, la jerarquía de puestos en una organización, etc.

Bosque y árboles

Un bosque es un gráfico que no contiene ciclos y un bosque conectado es un árbol. Por ejemplo, la figura 7.7 muestra un bosque con tres componentes, cada uno de los cuales es un árbol.



Fig. 7.7 Forest with three trees

Tenga en cuenta que los árboles y los bosques son gráficos simples. La siguiente terminología pertenece a los árboles.

Árbol dirigido

Un grafo dirigido acíclico es un árbol dirigido.

Raíz

Un árbol dirigido tiene un nodo llamado raíz, con grado cero, mientras que para todos los demás nodos, el grado inferior es 1.

Nodo terminal (nodo hoja)

En un árbol dirigido, cualquier nodo que tenga un grado cero es un nodo terminal. **El nodo terminal** también se denomina nodo hoja (o nodo externo).

Nodo de rama (nodo interno)

Todos los demás nodos cuyo grado de salida no es cero se denominan nodos de rama. Nivel de nodo El nivel de cualquier nodo es la longitud de su ruta desde la raíz. El nivel de la raíz de un árbol dirigido es cero, mientras que el nivel de cualquier nodo es igual a su distancia desde la raíz. La distancia desde la raíz es el número de bordes que se deben atravesar para llegar a la raíz.

Vamos a Arboles



Arboles | Trees

Introducción.

Arboles| C++
Introducción.

En ciencias de la computación, un **árbo**l es **una estructura de datos** ampliamente utilizada que emula una estructura de árbol con un conjunto de nodos vinculados. Los árboles se utilizan popularmente en la programación de computadoras. Se pueden utilizar para mejorar los tiempos de búsqueda de bases de datos (árboles de búsqueda binarios, árboles AVL, árboles rojos-negros), en programación de juegos (árboles minmax, árboles de decisión, árboles de ruta), programación de gráficos 3D (árboles binarios, cuadratos, octrees), lenguajes de scripting aritméticos (árboles de precedencia aritmética), compresión de datos (árboles de Huffman) e incluso sistemas de Archivos (btrees, árboles indexados dispersos, árboles de trígonos).



Arboles | Introducción



Las estructuras de datos no lineales son capaces de expresar relaciones más complejas que las estructuras de datos lineales.
En general, siempre que la relación jerárquica entre los datos deba ser preservada, el árbol se utiliza.
Ejemplos bien conocidos de tales estructuras son los árboles genealógicos, jerarquía de puestos en la organización, etc. El árbol, una estructura de datos no lineal, es un medio para mantener y manipular datos en muchas aplicaciones.

Considera lo siguiente ejemplo:

El sistema operativo de un sistema informático organiza los archivos en directorios y subdirectorios. Los directorios también se denominan carpetas. El sistema operativo organiza carpetas y archivos utilizando una estructura de árbol como en la Fig. Una carpeta contiene otras carpetas (subcarpetas) y archivos. Esto se puede ver en el árbol dibujado en la Fig. Tenga en cuenta que la raíz aquí es de escritorio.



Los usos comunes de los árboles incluyen los siguientes:
1. Manipulación de datos jerárquicos
2. Hacer que la información sea fácil de buscar
3. Manipulación de listas ordenadas de datos
Un árbol es un gráfico llamado “directed acyclic graph”. Así que primero discutamos la terminología básica relacionada con los árboles.

Arboles | Arbol General



Árbol general Un árbol T se define **recursivamente** de la siguiente manera:
1. Un conjunto de elementos cero es un árbol, llamado árbol vacío (o árbol nulo).
2. Si T1, T2, ..., Tn son n árboles (para n> 0) y R (root) es un nodo, entonces el conjunto T que contiene R y los árboles T1, T2, ..., Tn son un árbol. Dentro de T, R se denomina raíz de T, y T1, T2, ..., Tn se denominan subárboles

El árbol de la figura 7.8 (a) es el árbol vacío ya que no hay nodos.
El árbol de la figura 7.8 (b) tiene un solo nodo, la raíz.
El árbol de la figura 7.8 (c) tiene 16 nodos.
El nodo raíz tiene cuatro subárboles. Las raíces de estos subárboles se denominan hijos de la raíz. Hay 16 nodos en el árbol, por lo que hay 15 subárboles no vacíos.
Los nodos sin subárboles se denominan nodos terminales o, más comúnmente, hojas.
Son 10 hojas del árbol de la figura 7.8 (c).

El grado de un nodo es el número de subárboles que tiene. Por tanto, el grado de los nodos de la figura 7.8 (c) varía de cero a cuatro. Por definición, el grado de cada nodo hoja es cero. El grado de un árbol es el grado máximo de un nodo en el árbol. Como el árbol de la figura 7.8 (a) no tiene nodos, no hay un grado máximo de un nodo y el grado del árbol no está definido. El árbol de la figura 7.8 (b) tiene grado cero y el árbol de la figura 7.8 (c) tiene grado cuatro. Dado que las relaciones familiares se pueden modelar como árboles, a menudo llamamos a la raíz de un árbol (o subárbol) padre y a las raíces de los subárboles hijos. En consecuencia, los hijos del mismo nodo se denominan hermanos. La ventaja de la relación entre un padre y sus hijos es que un borde dirigido (o simplemente un borde) se extiende desde un padre a sus hijos.

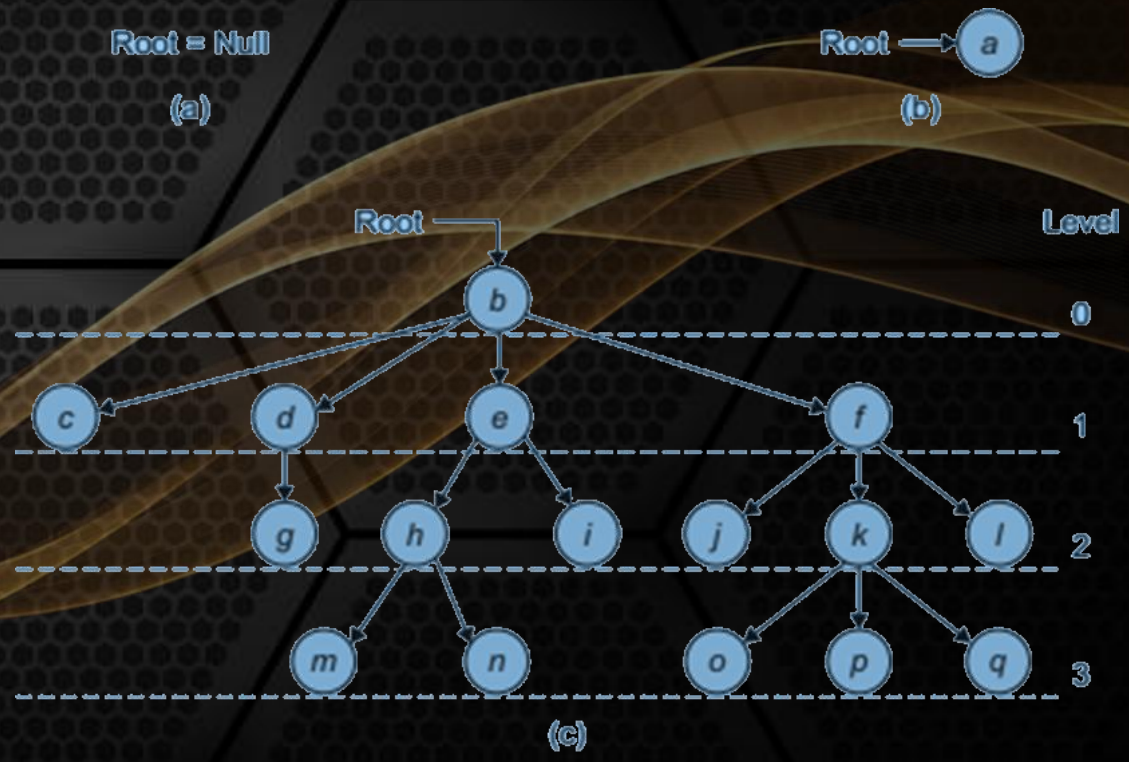
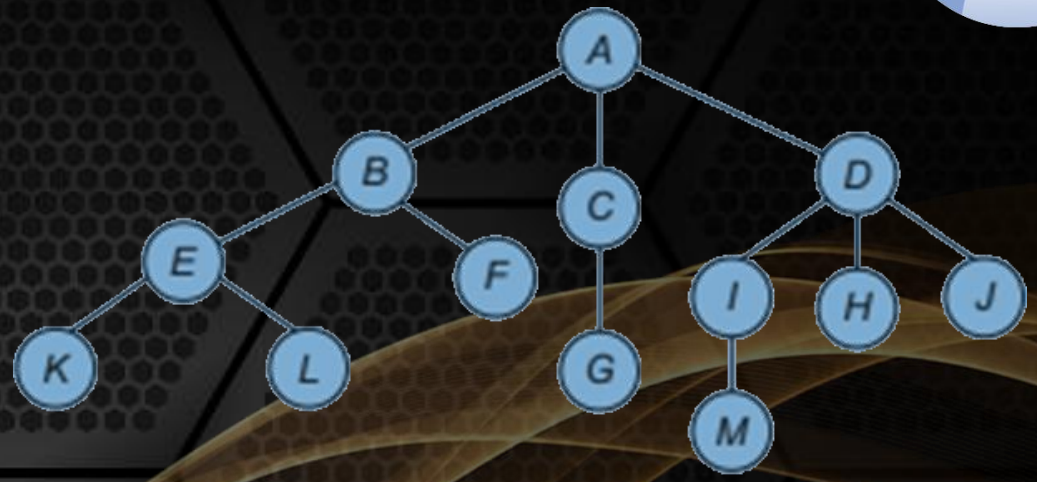


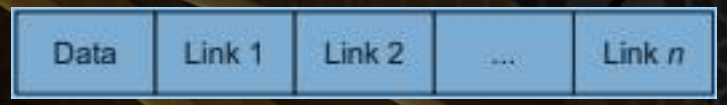
Fig. 7.8 Degree of a tree (a) Empty tree—degree undefined (b) Tree with a single node—degree 0 (c) Tree of height 3—degree 4

Arboles | Representación de un Arbol

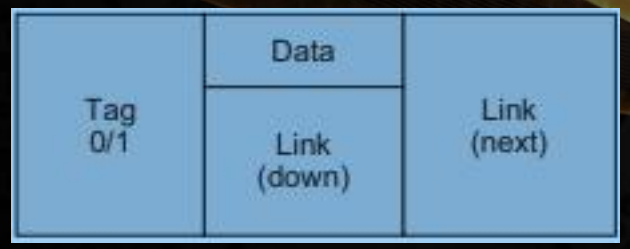


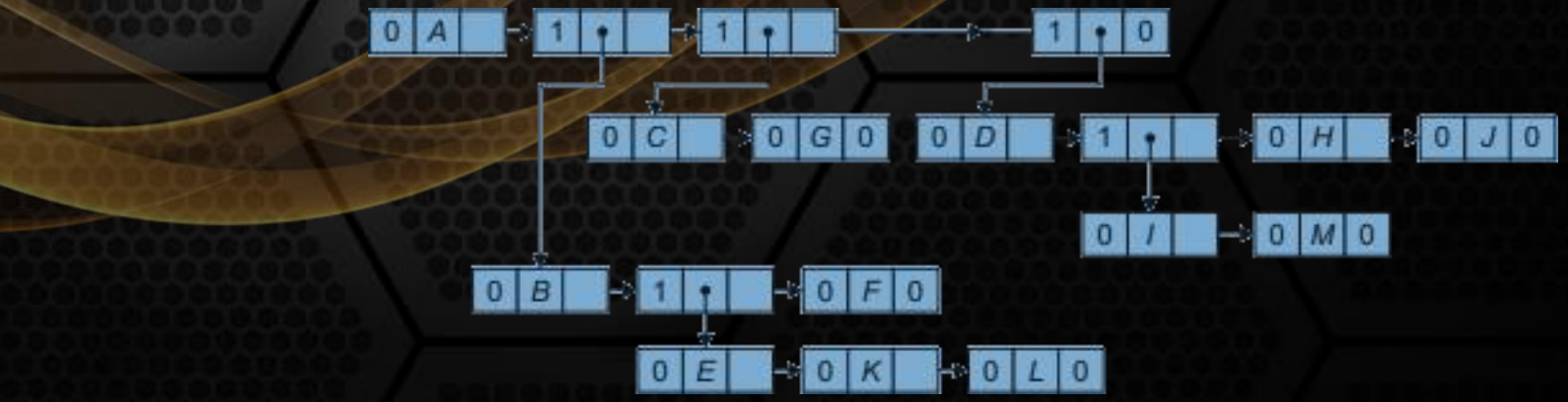
Para la Representación de un árbol en “general” Podemos utilizar una organización secuencial o una organización vinculada para representar un árbol. Si deseamos utilizar una lista enlazada generalizada, entonces un nodo debe tener un número variable de campos dependiendo del número de ramas. Sin embargo, es más sencillo utilizar algoritmos para los datos en los que el tamaño del nodo es fijo.





Para un nodo de tamaño fijo, podemos usar un nodo con datos y campos de puntero como en una lista enlazada generalizada.



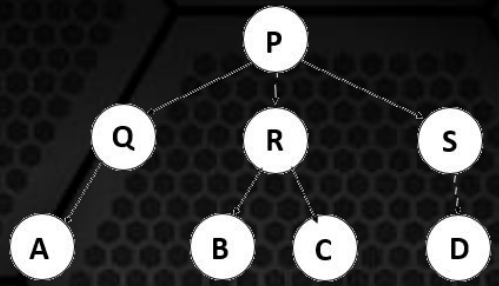


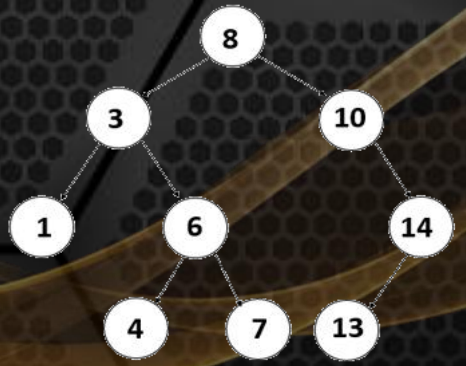
Arboles | Tipos de Arboles

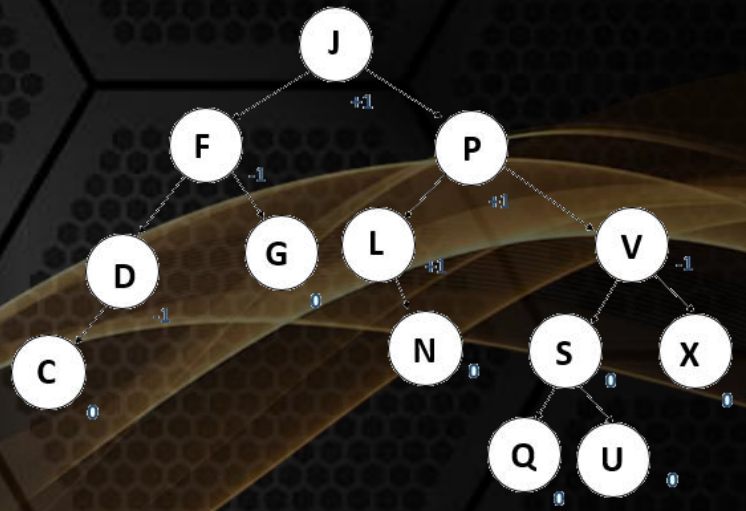


Algunos tipos importantes de árboles.

1. Árbol libre . Free tree
2. Árbol enraizado Rooted tree
3. Árbol ordenado Ordered tree
4. Árbol regular Regular tree
5. Árbol binario Binary tree
6. Árbol completo Complete tree
7. Árbol de posición Position tree







Veamos una Descripción

Arboles | Arbol General

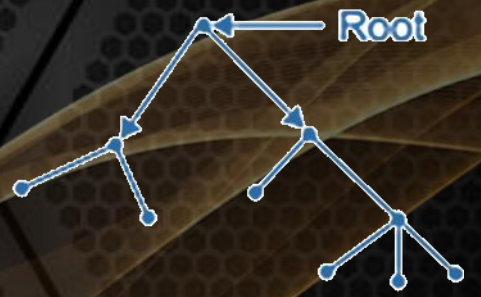
Árbol libre

Un árbol libre es un gráfico acíclico conectado. Es un gráfico no dirigido. No tiene ningún nodo designado como raíz. Como está conectado, se puede llegar a cualquier nodo desde cualquier otro nodo a través de una ruta única. El árbol de la figura es un ejemplo de árbol libre.



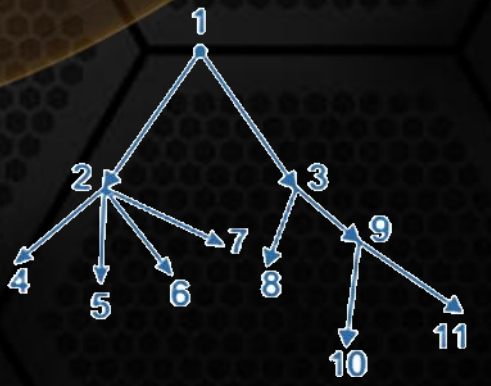
Árbol enraizado

A diferencia del árbol libre, un árbol enraizado es un gráfico dirigido donde un nodo se designa como raíz, cuyo grado entrante es cero, mientras que para todos los demás nodos, el grado entrante es uno .



Árbol ordenado

En muchas aplicaciones, el orden relativo de los nodos en cualquier nivel particular adquiere cierta importancia. Es fácil imponer un orden en los nodos en un nivel refiriéndose a un nodo en particular como el primer nodo, a otro nodo como el segundo, y así sucesivamente. Este orden se puede realizar de izquierda a derecha Al igual que los nodos en cada nivel, podemos prescribir un orden a los bordes. Si en un árbol dirigido, se prescribe un orden de un nodo en cada nivel, entonces dicho árbol se llama árbol ordenado.





Arboles | Arbol General



Árbol regular

Un árbol en el que cada vértice del nodo de rama tiene el mismo grado externo se denomina árbol regular. Si en un árbol dirigido, el grado exterior de cada nodo es menor o igual que m, entonces el árbol se llama árbol m-ario. Si el grado exterior de cada nodo es exactamente igual a m (los nodos de las ramas) o cero (los nodos de las hojas), entonces el árbol se denomina árbol m-ario regular.



Árbol completo

Un árbol con n nodos y de profundidad k está completo si y solo si sus nodos corresponden a los nodos que están numerados de 1 a n en el árbol completo de profundidad k. Un árbol binario de altura h está completo si y solo si se cumple una de las siguientes condiciones:

1. Está vacío.
2. Su subárbol izquierdo está completo de altura h-1 y su subárbol derecho está completamente lleno de altura h-2.
3. Su subárbol izquierdo está completamente lleno de altura h-1 y su subárbol derecho está completo de altura h-1. Un árbol binario está completamente lleno si tiene una altura hy tiene (2h 1 - 1) nodos.

Árbol binario

Un árbol binario es una forma especial de árbol m-ario. Dado que un árbol binario es importante, se utiliza con frecuencia en diversas aplicaciones de la informática.
Hemos definido un árbol m-ario (árbol general). Un árbol binario es un árbol de posición m-aria cuando m = 2. En un árbol binario, ningún nodo tiene más de dos hijos







Arboles | Arbol General

Árbol binario completo

Se dice que un árbol binario es un árbol binario completo si todos sus niveles, excepto el último, tienen el número máximo de nodos posibles, y todos los nodos del último nivel aparecen lo más a la izquierda posible. En un árbol binario completo, todos los nodos de hojas están en el último y el penúltimo nivel, y los niveles se llenan de izquierda a derecha. La figura 7.16 es un árbol binario completo.



Fig. 7.16 Complete binary tree

Árbol binario sesgado hacia la izquierda

Si el subárbol derecho falta en todos los nodos de un árbol, lo llamamos árbol sesgado hacia la izquierda (figura 7.17). Si falta el subárbol izquierdo en todos los nodos de un árbol, lo llamamos subárbol derecho (figura 7.18).

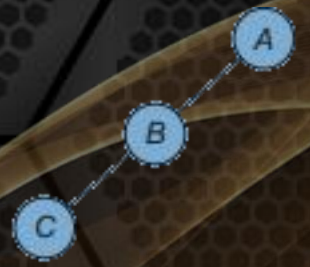


Fig. 7.17 Left skewed tree

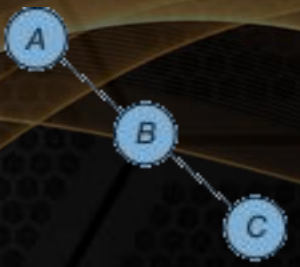


Fig. 7.18 Right skewed tree

Las aplicaciones como la cola de prioridad y la ordenación del montón utilizan el árbol binario completo



Arboles | Arbol General



Árbol estrictamente binario

Si cada nodo no terminal en un árbol binario consta de subárboles izquierdo y derecho no vacíos, entonces dicho árbol se denomina árbol estrictamente binario. En la figura 7.19, los nodos no vacíos D y E tienen subárboles izquierdo y derecho. Estos árboles de expresión se conocen como árboles estrictamente binarios.



Fig. 7.19 Strictly binary tree

Árbol binario extendido

Un árbol binario T en el que cada nodo tiene cero o dos hijos se denomina árbol binario extendido. Los nodos con dos hijos se denominan nodos internos y los que no tienen hijos se denominan nodos externos. Los árboles se pueden convertir en árboles extendidos agregando un nodo (Fig. 7.20).

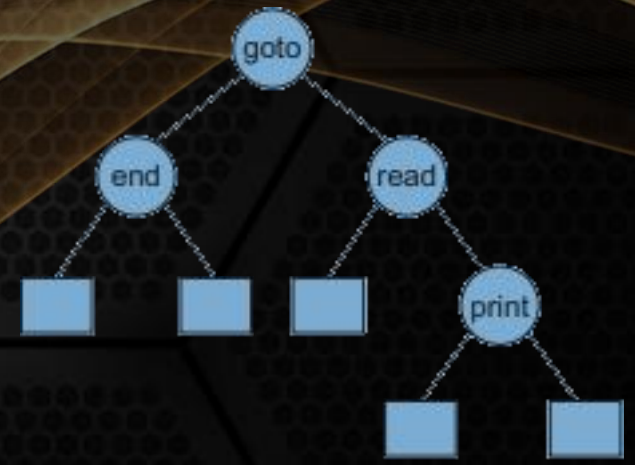


Fig. 7.20 Extended binary tree

Árbol de posición

Un árbol de posición, también conocido como árbol de sufijos, es aquel que representa los sufijos de una cadena S y dicha representación facilita que las operaciones de cadena se realicen más rápidamente. Los bordes de un árbol de este tipo están etiquetados con cadenas, de modo que cada sufijo de S corresponde exactamente a un camino desde la raíz del árbol hasta un nodo de hoja. El requisito de espacio y tiempo es lineal en la longitud de S. Después de su construcción, se pueden realizar varias operaciones rápidamente, como ubicar una subcadena en S, ubicar una subcadena si se permite un cierto número de errores, ubicar coincidencias para una expresión regular patrón, etc.

Arboles Binarios

Arboles| Arboles Binarios
Introducción.

Binary Tree

3



Arboles | Arbol Binario



Un árbol binario es un árbol de grado 2 en el cual sus hijos se identifican como subárbol izquierdo y subárbol derecho. Por lo tanto, cada nodo almacena información y las direcciones de sus descendientes (máximo 2). Es un tipo de árbol muy usado, ya que saber el número máximo de hijos que puede tener cada nodo facilita las operaciones sobre ellos. La figura debajo presenta el esquema de un nodo de un árbol binario.

La Estructura de abajo presenta un ejemplo de uso de un árbol binario. En este caso, la estructura se emplea para almacenar el árbol genealógico de María. En cada nodo se guarda la información de los ancestros de María y los arcos indican la relación entre ellos

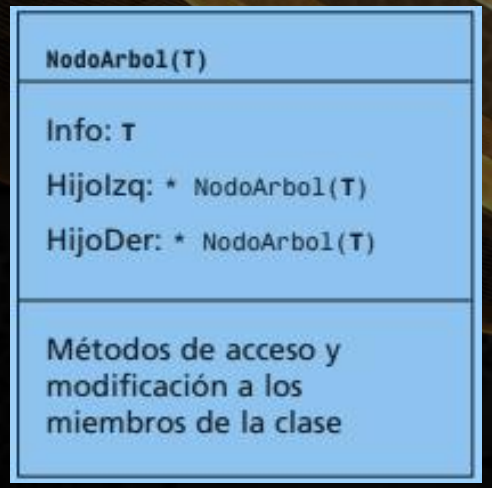
Dirección Subárbol Izquierdo	Información	Dirección Subárbol Derecho



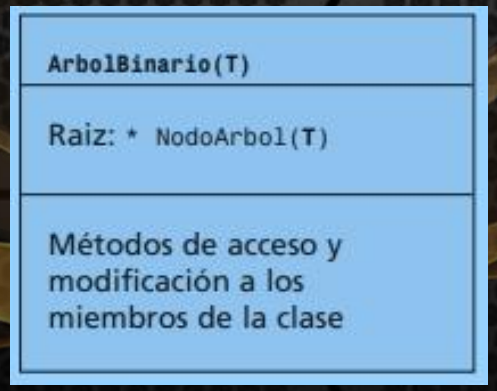
Arboles | Arbol Binario



La característica de este tipo de árbol (cada nodo tiene máximo 2 hijos) se puede aprovechar para organizar la información. Retomando el ejemplo de la figura 7.3, se puede establecer que los hijos izquierdos representen los ascendientes femeninos de María, mientras que los hijos derechos los ascendientes masculinos. Así, la mamá de María es Juana y su abuela materna es Inés. El papá de María es José. A su vez, la mamá de José es Felisa y su papá es Pedro. **La implementación más efectiva de los árboles binarios es por medio de memoria dinámica**, obteniendo así una estructura dinámica. Las figuras 7.4 y 7.5 presentan las plantillas de la clase **NodoArbol** y de la clase **ArbolBinario** respectivamente. Se usan plantillas para dar mayor generalidad a la solución. La clase **NodoArbol** tiene tres atributos, uno que representa la información a almacenar por lo que se define de tipo T, y otros dos que representan la dirección del hijo izquierdo y del hijo derecho respectivamente, por lo que se declaran como punteros a objetos de la misma clase. Por su parte, la clase **ArbolBinario** tiene un único atributo que representa la dirección del primer elemento del árbol (root o la raíz) por lo cual es de tipo puntero a un objeto de tipo **NodoArbol**



Clase NodoArbol



Clase ArbolBinario

figuras 7.4 y 7.5



figura 7.3

Arboles | Arbol Binario

A continuación se presenta el código en lenguaje C++ correspondiente a la definición de las plantillas de las clases NodoArbol y ArbolBinario.

```
/*Prototipo de la plantilla de la clase ArbolBinario. De esta manera,
en la clase NodoArbol se podrá hacer referencia a ella. */
template < class T>
    class ArbolBinario;
/*Declaración de la clase NodoArbol. Cada nodo almacena la información
(que es la razón de ser de la estructura tipo árbol) y las direcciones
de sus hijos izquierdo y derecho. En la sección pública se establece la
relación de amistad entre esta clase y la clase ArbolBinario para que los
métodos de esta última puedan tener acceso a sus miembros privados. */
template < class T>
    class NodoArbol
    {
       private:
        T Info;
        NodoArbol<T> *HijoIzq;
        NodoArbol<T> *HijoDer;

       public:
         NodoArbol();
        T RegresaInfo();
        friend class ArbolBinario<T> ;
    };
/*Declaración del método constructor por omisión. Inicializa las ligas
a los subárboles con el valor de NULL, indicando que están vacías. */
template < class T>
    NodoArbol<T>::NodoArbol()
    {
        HijoIzq = NULL;
        HijoDer = NULL;
    }
```

```
/*Método que permite conocer la información almacenada en el nodo. */
template < class T>
    T NodoArbol<T>::RegresaInfo()
    {
        return Info;
    }

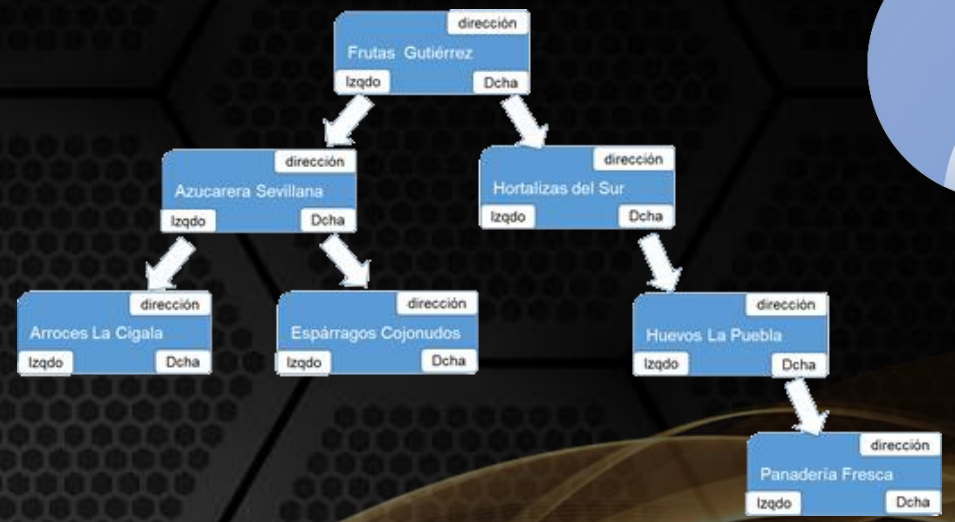
//-----------------------------------------------------------------------
/*Declaración de la clase ArbolBinario. Su atributo es un puntero al
nodo raíz. */
template < class T>
    class ArbolBinario
    {
        private:
            NodoArbol<T> *Raiz;
        public:
            ArbolBinario();
        /*En esta sección se declaran los métodos de acceso y
        ➥modificación a los miembros de la clase. */
    };
/*
Declaración del método constructor. Inicializa el puntero a la raíz
con el valor NULL, indicando que el árbol está vacío (no tiene nodos).
*/
template < class T>
    ArbolBinario<T>::ArbolBinario()
    {
        Raiz = NULL;
    }
```

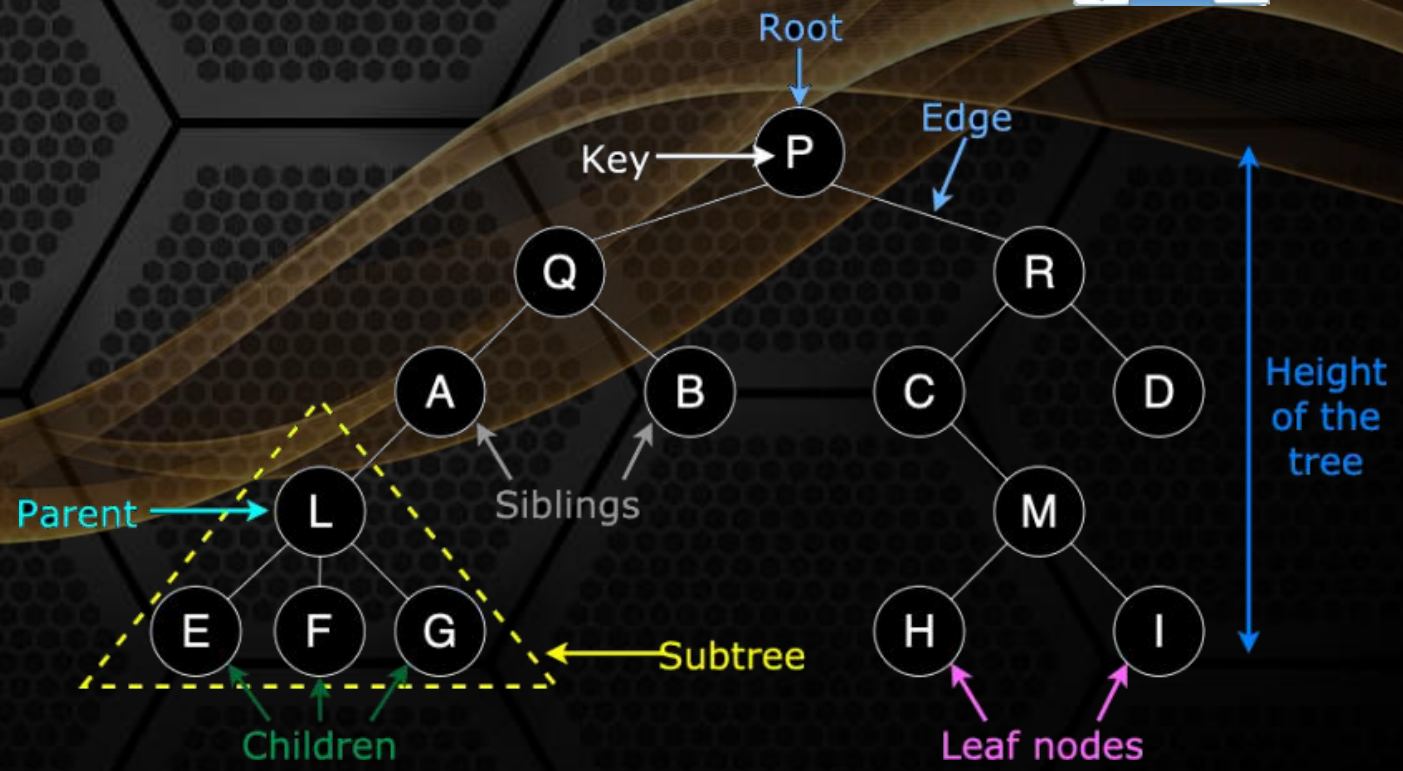


Arboles | Arbol Binario

La clase NodoArbol se utiliza para representar un nodo de un árbol binario, por lo tanto se incluyen tres atributos: uno para almacenar información de cualquier tipo (tipo T) y los otros dos para almacenar la dirección de los subárboles izquierdo y derecho respectivamente, los cuales son punteros a objetos de la misma clase.

La sección pública contiene tres miembros (podría tener más o menos), dependiendo de la definición de la clase que se haga. Estos elementos son: el método constructor, un método que facilita (a usuarios externos a la clase) conocer la información guardada, y la declaración de amistad con la clase ArbolBinario. Esta última declaración permite que los métodos de la clase amiga tengan acceso a sus miembros privados y protegidos. A partir de la clase NodoArbol se define la clase ArbolBinario, la cual está formada por un atributo único (tipo puntero a un objeto NodoArbol) que representa el puntero al nodo raíz del árbol binario. Este puntero permite el acceso a todos los elementos del árbol ya que la raíz tiene la dirección de sus dos hijos, éstos, la dirección de sus respectivos hijos y así hasta llegar a nodos terminales. En la sección pública se declaran los métodos necesarios para tener acceso a los atributos, y de esta manera manipular la información almacenada.





Arboles | Operaciones | Crear

En esta sección se estudian las operaciones de creación y recorrido de un árbol binario.
La primera hace referencia a crear una estructura que responda a las características analizadas e ir almacenando información en cada uno de los nodos
La segunda permite visitar todos los nodos de un árbol sin repetir ninguno, aprovechando el conocimiento que se tiene acerca de la estructura.

La **creación** de un árbol binario se lleva a cabo a partir de la raíz. Se crea un nodo y se almacena su información. Posteriormente se pregunta si dicho nodo tiene hijo izquierdo, si la respuesta es afirmativa, entonces se invoca nuevamente el método pero ahora con el subárbol izquierdo. El proceso se repite con cada nodo hasta llegar a las hojas. Luego, se hace lo mismo para crear cada uno de los subárboles derechos. Se utiliza la instrucción new() para asignar un espacio de memoria de manera dinámica. A continuación se presenta el método para llevar a cabo la secuencia de pasos descrita

```
/*Plantilla del método que crea un árbol binario. Recibe como parámetro un ap
untador a un subárbol. La primera vez es la raíz del árbol la cual se inicial
izó con el valor NULL, indicando que el árbol está vacío. */
template < class T>
    void ArbolBinario<T>::CreaArbol(NodoArbol<T> *Apunt)
    {
        char Resp;
        /*Se crea un nodo. */
        Apunt = new NodoArbol<T> ;
        cout << ”\n\n Ingrese la información a almacenar: ”;
        cin >> Apunt-> Info;
        cout << ”\n \n” << Apunt-> Info << ”¿Tiene hijo izquierdo(S / N) ? ”;
        cin >> Resp;
        if (Resp == ‘s’)
        {
            /*Se invoca al método con el subárbol izquierdo.
              Se usa la definición recursiva de un árbol. */

            CreaArbol(Apunt- > HijoIzq);
            Apunt- > HijoIzq = Raiz;
        }

        cout << ”\n \n” << Apunt-> Info  << ”¿Tiene hijo derecho(S / N) ? ”;
        cin >> Resp;
        if (Resp == ‘s’)
        {
            /*Se invoca al método con el subárbol derecho.
              Se usa la definición recursiva de un árbol. */

            CreaArbol(Apunt- > HijoDer);
            Apunt- > HijoDer = Raiz;
        }

        Raiz = Apunt;
    }
```



Arboles | Árbol Binario | Recorrido



El **Recorrido** de un árbol binario consiste en visitar todos sus nodos una sola vez. Por lo tanto, podrá hacerse (aprovechando las características de la estructura del árbol) de tres maneras diferentes: visitando la raíz, el hijo izquierdo y el hijo derecho, o visitando el hijo izquierdo, la raíz y el hijo derecho, o bien, visitando el hijo izquierdo, el hijo derecho y la raíz. En los tres casos, la regla se aplica hasta llegar a las hojas. Estos métodos se conocen con el nombre de preorden, inorden y postorden respectivamente

Considerando el árbol binario de la figura 7.6, el resultado de los tres recorridos es el siguiente.

- Preorden: 304 – 550 – 143 – 2020 – 1995 – 876 – 609 – 300
- Inorden: 143 – 550 – 2020 – 304 – 876 – 1995 – 609 – 300
- Postorden: 143 – 2020 – 550 – 876 – 300 – 609 – 1995 – 304

Preorden	Inorden	Postorden
1. Visita la raíz	1. Recorre el subárbol izquierdo	1. Recorre el subárbol izquierdo
2. Recorre el subárbol izquierdo	2. Visita la raíz	2. Recorre el subárbol derecho
3. Recorre el subárbol derecho	3. Recorre el subárbol derecho	3. Visita la raíz



La naturaleza **recursiva** de los métodos permite lograr la impresión de todos los nodos

Arboles | Arbol Binario | ejemplo

```
/*Método que realiza el recorrido preorden de un árbol binario. Se usa el recorrid
o para imprimir la información almacenada en cada uno de sus nodos. Recibe como par
ámetro el nodo a visitar. La primera vez es la raíz del árbol, luego será la raíz d
el subárbol izquierdo y la raíz del subárbol derecho y así hasta llegar a las hoja
s. */


template < class T>
    void ArbolBinario<T>::Preorden(NodoArbol<T> *Apunt)
    {
        if (Apunt)
        {
            cout << Apunt- > Info << ””;
            Preorden(Apunt- > HijoIzq);
            Preorden(Apunt- > HijoDer);
        }
    }
```

```
/*Método que realiza el recorrido inorden de un árbol binario. Se usa
el recorrido para imprimir la información almacenada en cada uno de sus
nodos. Recibe como parámetro el nodo a visitar. La primera vez es la
raíz del árbol, luego será la raíz del subárbol izquierdo y la raíz del
subárbol derecho y así hasta llegar a las hojas. */


template < class T>
    void ArbolBinario<T>::Inorden(NodoArbol<T> *Apunt)
    {
        if (Apunt)
        {
            Inorden(Apunt- > HijoIzq);
            cout << Apunt- > Info << ””;
            Inorden(Apunt- > HijoDer);
        }
    }
```

```
/*Método que realiza el recorrido postorden de un árbol bina
rio. Se usael recorrido para imprimir la información almacen
ada en cada uno de susnodos. Recibe como parámetro el nodo
a visitar. La primera vez es la
raíz del árbol, luego será la raíz del subárbol izquierdo y
la raíz delsubárbol derecho y así hasta llegar a las hojas.
*/

template < class T>
    void ArbolBinario<T>::Postorden(NodoArbol<T> *Apunt)
    {
        if (Apunt)
        {
            Postorden(Apunt- > HijoIzq);
            Postorden(Apunt- > HijoDer);
            cout << Apunt- > Info << ””;
        }
    }
```



Arboles | Arbol Binario



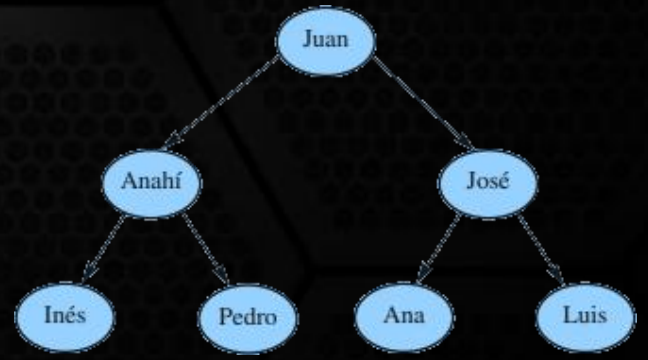
En los tres métodos, la instrucción de imprimir se da sobre el contenido de la raíz.
La naturaleza **recursiva** de los métodos permite lograr la impresión de todos los nodos. Las instrucciones que forman cada uno de los métodos son las mismas, lo único que cambia es el orden en el cual se ejecutan. Si se analiza el recorrido preorden con el árbol de la figura 7.6, se puede observar que primero se imprime el número 304, luego se invoca el método con el subárbol izquierdo, quedando pendiente el recorrido con el subárbol derecho (internamente se guardan en una pila las instrucciones pendientes de ejecutar). Lo mismo sucede cuando llega con el subárbol izquierdo, imprime el valor 550 e invoca el método con su subárbol izquierdo y deja pendiente el recorrido con su subárbol derecho. Una vez agotado el lado izquierdo, pasa al lado derecho del último nodo visitado. Se van tomando de la pila todos los subárboles derechos que quedaron pendientes de visitar y se van recorriendo.

Como consecuencia, el primer número (correspondiente a un subárbol derecho) que se imprime es el 2020, luego el 1995 y como este último tiene subárbol izquierdo, entonces se invoca al método con éste (876). Se repite el proceso hasta que ya no queden nodos a visitar. En un árbol binario también pueden realizarse otras operaciones como buscar, insertar o eliminar un dato en un árbol ya generado



El programa 7.1 presenta una aplicación de árboles binarios. El programa imprime los datos de todos los ascendientes femeninos de un individuo, tanto de la rama materna como de la paterna. Se utiliza un objeto tipo árbol binario para almacenar los datos de los ascendientes de una persona, es decir, su árbol genealógico. En la raíz de cada subárbol izquierdo se almacena la información de un ascendiente femenino, mientras que en la raíz de cada subárbol derecho se guarda la información de un ascendiente masculino. Considerando el árbol de la figura 7.7, algunas de las relaciones familiares representadas son:

Anahí es mamá de Juan
José es papá de Juan
Inés es mamá de Anahí
Pedro es papá de Anahí
Ana es mamá de José
Luis es papá de José



Arboles | Arbol Binario





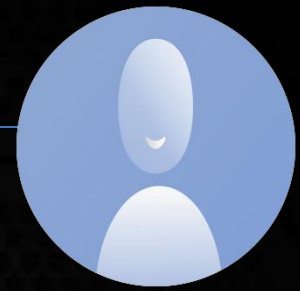
Btree_Full.cpp

Arboles| Arboles Binarios Búsqueda.

Binary Search Tree



Árboles binarios de búsqueda

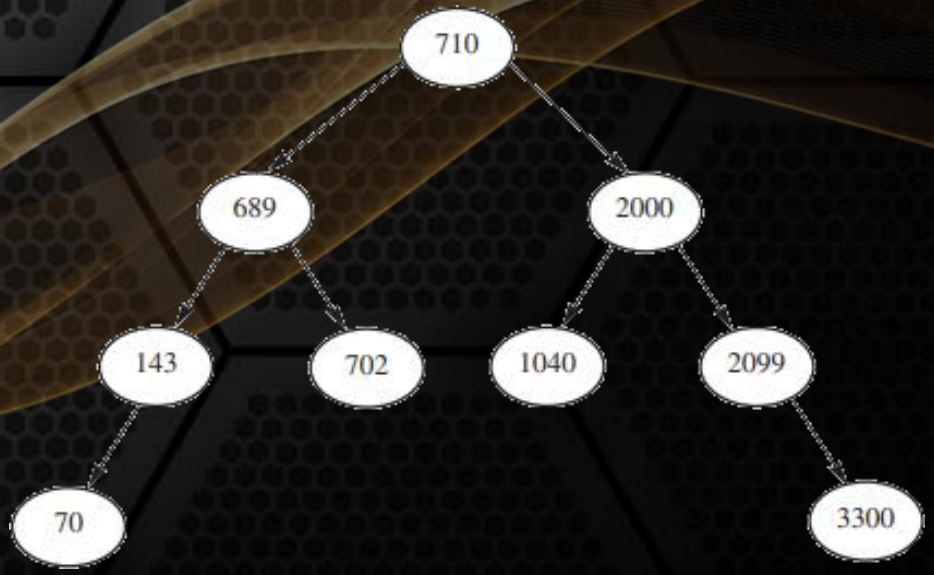


Un árbol binario de búsqueda se caracteriza porque la información de cada nodo es mayor que la información de cada uno de los nodos que están en su subárbol izquierdo y menor que la almacenada en los nodos que están en su subárbol derecho. La figura 7.8 presenta un ejemplo de árbol binario de búsqueda. Observe que todos los valores que están a la izquierda del 710 son menores que él. A su vez, los que están a su derecha son mayores. La misma regla se aplica en todos los nodos.

El recorrido inorden de un árbol binario de búsqueda genera una lista ordenada de manera creciente de todos sus elementos. Tomando el árbol de la figura 7.8, este recorrido proporcionaría los elementos en el siguiente orden:

70 – 143 – 689 – 702 – 710 – 1040 – 2000 – 2099 – 3300

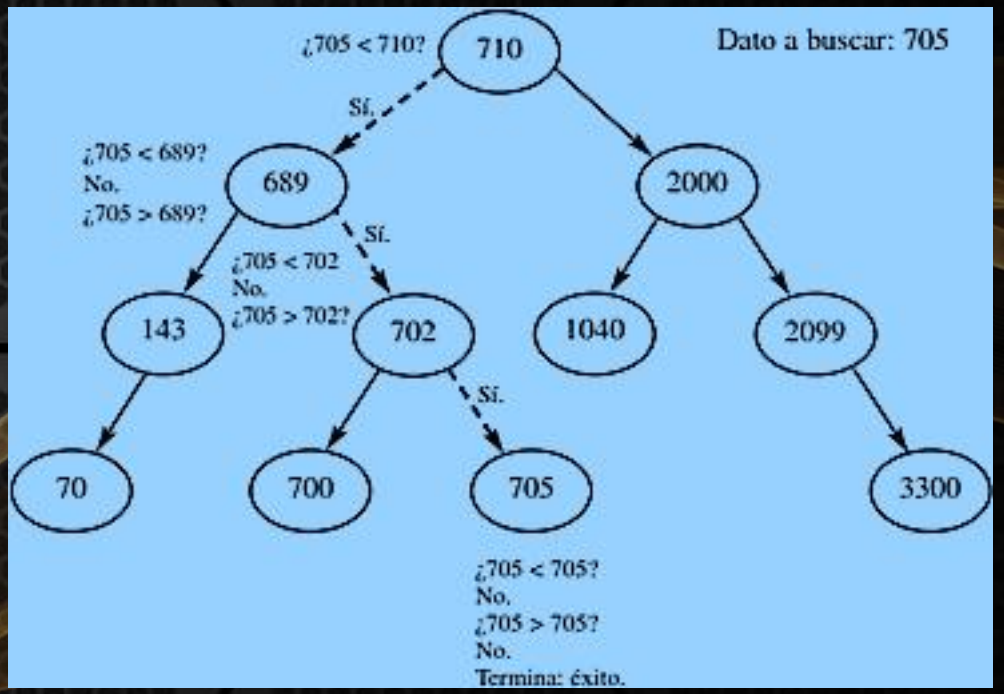
El orden que existe entre la información almacenada en el árbol facilita la operación de búsqueda de cualquiera de sus elementos. A continuación se analizarán las operaciones de búsqueda, inserción y eliminación en árboles binarios de búsqueda.



Árboles binarios de búsqueda | Operación de búsqueda

Para llevar a cabo la búsqueda de un elemento en un árbol binario de búsqueda se procede de la siguiente manera:

1. Se evalúa si el nodo visitado (la primera vez es la raíz) está definido.
2. Si la respuesta es afirmativa, se pregunta si el dato buscado es menor que el dato visitado.
    1. Si la respuesta es afirmativa, se procede a buscar el dato en el subárbol izquierdo.
    2. Si la respuesta es negativa, se pregunta si el dato buscado es mayor que el dato visitado.
        1. Si la respuesta es afirmativa, se procede a buscar el dato en el subárbol derecho.
        2. Si la respuesta es negativa, la búsqueda termina exitosamente. El dato fue encontrado.
3. Si la respuesta a la pregunta 1 es negativa, entonces termina la búsqueda con un fracaso.



Árboles binarios de búsqueda - Ejemplo



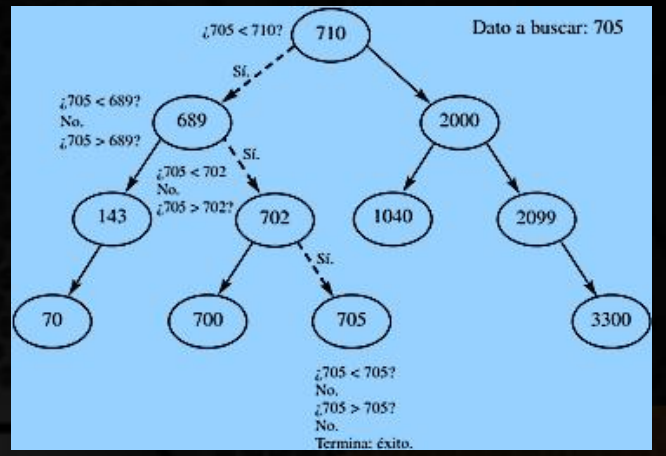
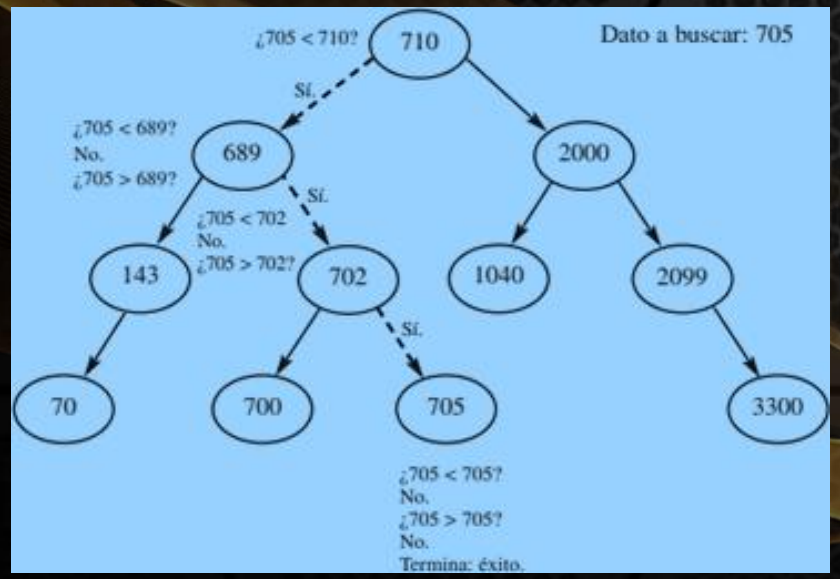


TABLA 7.1 Operación de búsqueda en un árbol binario de búsqueda

1 Se evalúa si el nodo visitado (710) está definido. En este caso sí lo está.
2 Se evalúa si el dato buscado (705) es menor que la información almacenada (710) en el nodo visitado. En este caso sí lo es.
3 Se invoca el método con el subárbol izquierdo.
4 Se evalúa si el nodo visitado (689) está definido. En este caso sí lo está.
5 Se evalúa si el dato buscado (705) es menor que la información almacenada (689) en el nodo visitado. En este caso no lo es.
6 Se evalúa si el dato buscado (705) es mayor que la información almacenada (689) en el nodo visitado. En este caso sí lo es.
7 Se invoca el método con el subárbol derecho.
8 Se evalúa si el nodo visitado (702) está definido. En este caso sí lo está.
9 Se evalúa si el dato buscado (705) es menor que la información almacenada (702) en el nodo visitado. En este caso no lo es.
10 Se evalúa si el dato buscado (705) es mayor que la información almacenada (702) en el nodo visitado. En este caso sí lo es.
11 Se invoca el método con el subárbol derecho.
12 Se evalúa si el nodo visitado (705) está definido. En este caso sí lo está.
13 Se evalúa si el dato buscado (705) es menor que la información almacenada (705) en el nodo visitado. En este caso no lo es.
14 Se evalúa si el dato buscado (705) es mayor que la información almacenada (705) en el nodo visitado. En este caso no lo es.
15 La búsqueda termina con éxito. El dato fue encontrado

Árboles binarios de búsqueda - Ejemplo

Como se puede deducir de los pasos presentados, el número de comparaciones se reduce a la mitad en cada nodo visitado. No se requiere visitar todos los nodos. El método que implementa esta operación es el siguiente:




```
/* Método que busca un dato en un árbol binario de búsqueda. Recibe como
parámetros un apuntador, que es la dirección del nodo a visitar (la primera vez es
el apuntador a la raíz) y el dato a buscar. Regresa como resultado la dirección
del nodo encontrado o el valor NULL, si la bús- queda termina con fracaso. */

template < class T >
  NodoArbol < T > * ArbolBinBus < T > ::Busqueda(NodoArbol < T > * Apunt, T Dato)
{
    if (Apunt)
      if (Dato < Apunt- > Info)
        return Busqueda(Apunt- > HijoIzq, Dato);
      else
    if (Dato > Apunt- > Info)
      return Busqueda(Apunt- > HijoDer, Dato);
    else
      return Apunt;
    else
      return NULL;
  }
```



Arboles| Arboles Binarios
Inserción.

Binary Search Insert



Árboles binarios | Inserción

Insertar un nuevo elemento en un árbol binario de búsqueda requiere buscar la posición que debe ocupar el nuevo nodo de tal manera que no altere el orden del árbol. En la solución que aquí se propone no se aceptan elementos repetidos. Pueden existir aplicaciones en las cuales sí se permita. Sin embargo, en esta solución, si se detecta que en el árbol ya está almacenado un valor igual al que se pretende insertar, la operación se interrumpe.

Los principales pasos para llevar a cabo esta operación son:

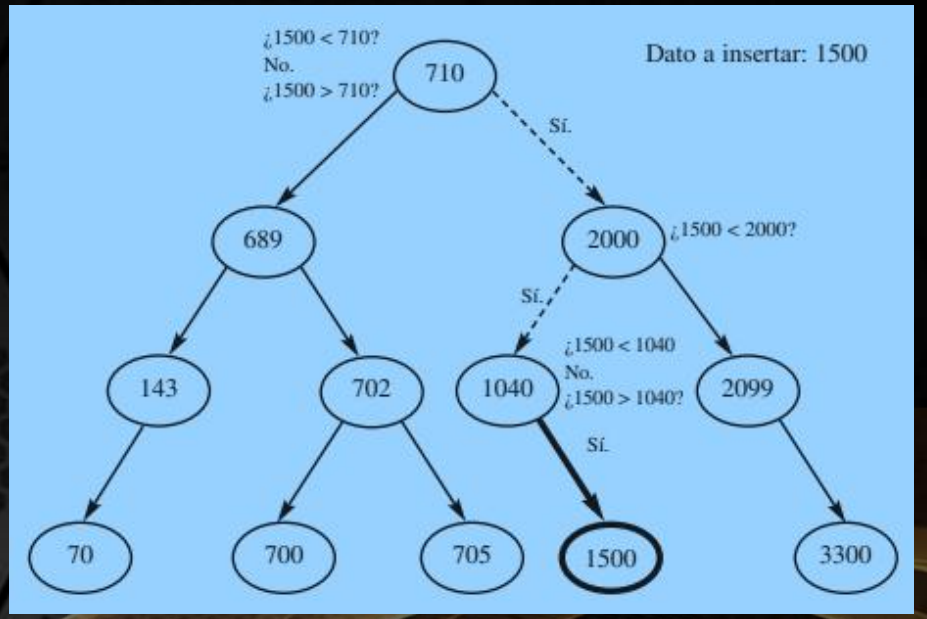
1) Se evalúa si el nodo (la primera vez es la raíz) visitado está definido.
2) Si la respuesta es afirmativa, se pregunta si el dato a insertar es menor que el dato visitado.
    1) Si la respuesta es afirmativa, se invoca el proceso de inserción con el subárbol izquierdo.
    2) Si la respuesta es negativa, se pregunta si el dato a insertar es mayor que el dato visitado
        1) Si la respuesta es afirmativa, entonces se invoca el proceso de inserción con el subárbol derecho.
        2) Si la respuesta es negativa, entonces el proceso de inserción termina sin haberse realizado, ya que no se permiten elementos repetidos.

3. Si la respuesta al paso 1 es negativa, se crea el nuevo nodo, se almacena la información y se establecen las ligas entre el nuevo nodo y su padre.



Árboles binarios | Inserción

Operación de inserción en un árbol binario de búsqueda



Suponga que en el árbol de la figura 7.10 se quiere insertar el valor 1500. Las líneas punteadas indican los nodos visitados hasta encontrar el adecuado (1040) para proceder a la inserción. El nodo en negritas es el nuevo elemento agregado al árbol y la línea (también en negritas) es la liga entre éste y su padre. Aplicando el algoritmo dado, se realiza la secuencia de operaciones que se muestra en la tabla 7.2.

1 Se evalúa si el nodo visitado (710) está definido. En este caso sí lo está.
2 Se evalúa si el dato a insertar (1500) es menor que la información almacenada (710) en el nodo visitado. En este caso no lo es.
3 Se evalúa si el dato a insertar (1500) es mayor que la información almacenada (710) en el nodo visitado. En este caso sí lo es.
4 Se invoca el método con el subárbol derecho.
5 Se evalúa si el nodo visitado (2000) está definido. En este caso sí lo está.
6 Se evalúa si el dato a insertar (1500) es menor que la información almacenada (2000) en el nodo visitado. En este caso sí lo es.
7 Se invoca el método con el subárbol izquierdo.
8 Se evalúa si el nodo visitado (1040) está definido. En este caso sí lo está.
9 Se evalúa si el dato a insertar (1500) es menor que la información almacenada (1040) en el nodo visitado. En este caso no lo es.
10 Se evalúa si el dato a insertar (1500) es mayor que la información almacenada (1040) en el nodo visitado. En este caso sí lo es.
11 Se invoca el método con el subárbol derecho.
12 Se evalúa si el nodo visitado (NULL) está definido. En este caso no lo está.
13 Se crea un nuevo nodo, se le asigna el valor 1500 y se establece la liga entre él y su padre (el nodo que almacena el número 1040). El proceso termina

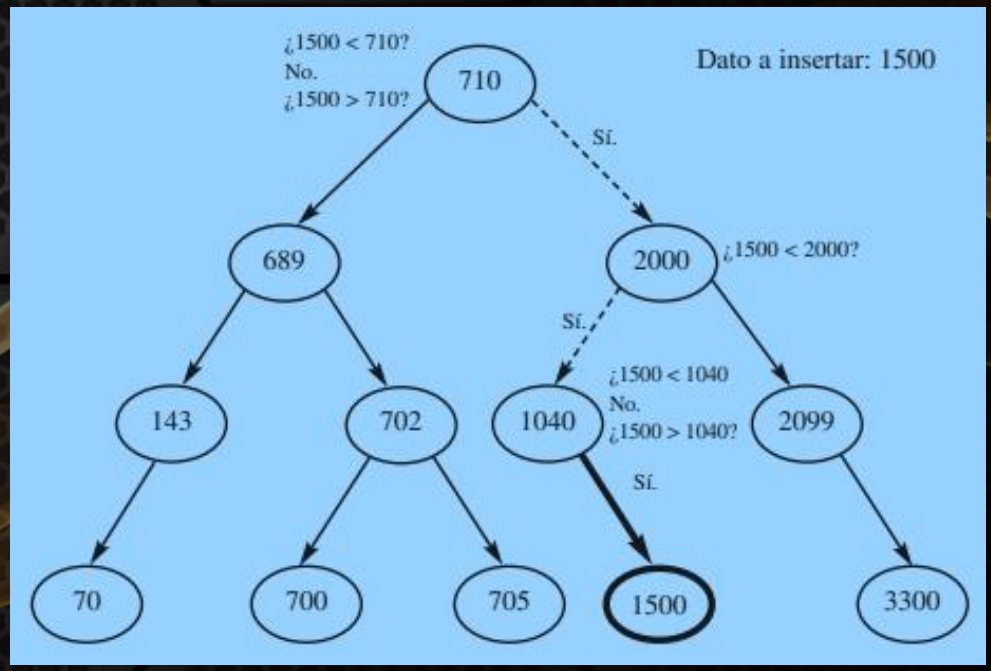


Árboles binarios | Inserción - ejemplo



```
/* Método que inserta un nodo en un árbol binario de búsqueda. Recibe como
parámetros un apuntador (la primer a vez es la raíz del árbol) y la información
que se quiere almacenar en el nuevo nodo. En esta implementación no se permite
que haya información duplicada en el árbol. */

template < class T >
  void ArbolBinBus < T > ::InsertaNodoSinRep(NodoArbol < T > * Apunt, T Dato) {
    NodoArbol < T > * ApAux;
    if (Apunt) {
      if (Dato < Apunt- > Info) {
        InsertaNodoSinRep(Apunt- > HijoIzq, Dato);
        Apunt- > HijoIzq = Raiz;
      } else
      if (Dato > Apunt- > Info) {
        InsertaNodoSinRep(Apunt- > HijoDer, Dato);
        Apunt- > HijoDer = Raiz;
      }
      Raiz = Apunt;
    } else {
      /* Se crea un nuevo nodo, se le asigna la información y se establecen
         las ligas entre los nodos correspondientes. */
      ApAux = new NodoArbol < T > ();
      ApAux- > Info = Dato;
      Raiz = ApAux;
    }
  }
```





Arboles| Arboles Binarios Eliminación.

Binary Search Delete

Árboles binarios | Eliminación

Para eliminar un elemento en un árbol binario de búsqueda se requiere buscar el valor deseado y quitar el nodo que lo contiene. Este último paso se lleva a cabo de maneras diferentes dependiendo si el nodo eliminado es terminal o no. Si se trata de una hoja, entonces se quita directamente. En otro caso, para no perder las ligas a sus descendientes, se debe reemplazar por el nodo que se encuentra más a la derecha del subárbol izquierdo o por el que se encuentra más a la izquierda del subárbol derecho. En la solución que se da en esta libro se usa el elemento que está más a la derecha del subárbol izquierdo. Los principales pasos para llevar a cabo esta operación son

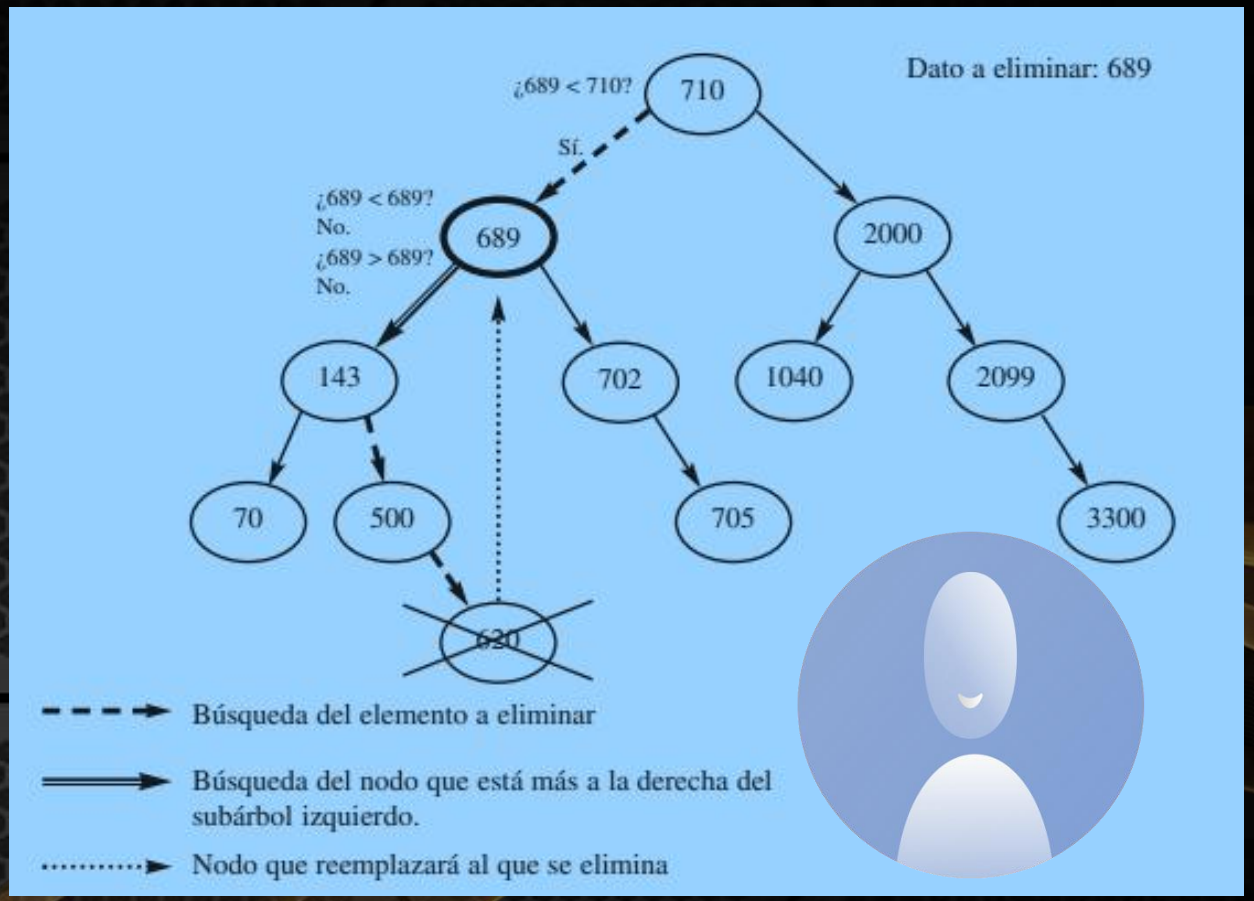
1. Se evalúa si el nodo visitado (la primera vez es la raíz) está definido.
2. Si la respuesta es afirmativa, entonces se pregunta si el dato a eliminar es menor que el dato visitado.
    1. 2.1. Si la respuesta es afirmativa, se invoca el proceso de eliminación con el subárbol izquierdo.
    2. 2.2. Si la respuesta es negativa, se pregunta si el dato a eliminar es mayor que el dato visitado
        1. 2.2.1. Si la respuesta es afirmativa, se invoca el proceso de eliminación con el subárbol derecho. 2
        2. 2.2.2. Si la respuesta es negativa, se elimina el nodo. Si es hoja, la eliminación es directa. Si tiene sólo un hijo se reemplaza por éste y si tiene dos se reemplaza por el que está más a la derecha del subárbol izquierdo. En estos dos últimos casos se libera el espacio de memoria del hijo, mientras que en el primero el correspondiente al nodo en cuestión
3. . Si la respuesta al paso 1 es negativa, entonces el dato no está en el árbol. El proceso de eliminación termina con fracaso.



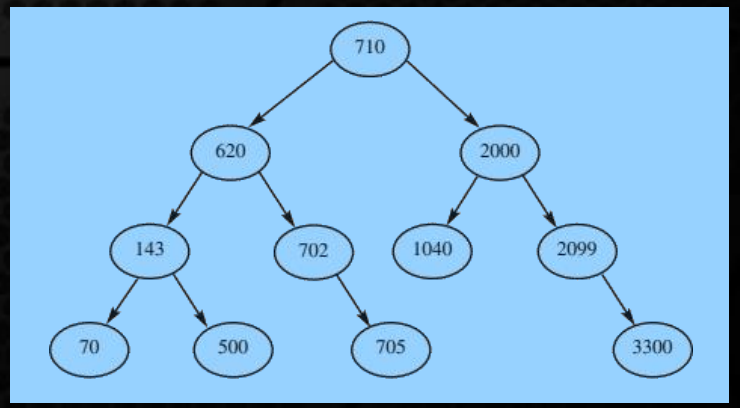
Árboles Binarios | Eliminación

Operación de eliminación en un árbol binario de búsqueda

Suponga que en el árbol binario de búsqueda de la figura 7.11 se quiere eliminar el valor 689. En (a) se muestra el camino que se sigue para llegar al nodo deseado y al nodo cuyo contenido reemplazará al 689. En (b) se presenta el árbol una vez que el nodo fue eliminado. La tabla 7.3 presenta las operaciones realizadas, siguiendo el algoritmo dado, para eliminar el 689



1 Se evalúa si el nodo visitado (710) está definido. En este caso sí lo está.
2 Se evalúa si el dato a eliminar (689) es menor que la información almacenada (710) en el nodo visitado. En este caso sí lo es.
3 Se invoca el método con el subárbol izquierdo.
4 Se evalúa si el nodo visitado (689) está definido. En este caso sí lo está.
5 Se evalúa si el dato a eliminar (689) es menor que la información almacenada (689) en el nodo visitado. En este caso no lo es.
6 Se evalúa si el dato a eliminar (689) es mayor que la información almacenada (689) en el nodo visitado. En este caso no lo es.
7 Se llegó al nodo que se quiere quitar. Como tiene dos hijos se reemplaza su contenido con el del nodo que está más a la derecha del subárbol izquierdo y se libera el espacio de memoria correspondiente al hijo. El proceso termina



Árboles binarios de búsqueda - Ejemplo

```
/* Método que elimina un nodo de un árbol binario de búsqueda.
Recibe como parámetro un apuntador (la primera vez es la raíz)
 y el  dato a eliminar. */


template < class T >
  void ArbolBinBus < T > ::EliminaNodo(NodoArbol < T > * Apunt, T Dato) {
    if (Apunt)
      if (Dato < Apunt- > Info) {
        EliminaNodo(Apunt- > HijoIzq, Dato);
        Apunt- > HijoIzq = Raiz;
      }
    else
    if (Dato > Apunt- > Info) {
      EliminaNodo(Apunt- > HijoDer, Dato);
      Apunt- > HijoDer = Raiz;
    } else {
      NodoArbol < T > * ApAux1, * ApAux2, * ApAux3;
      ApAux3 = Apunt;
      /* Encuentra el nodo que contiene el dato a eliminar. Verifica si tiene hijos. */
      if (!ApAux3- > HijoDer)
        if (!ApAux3- > HijoIzq)
          /* Si no tiene hijo derecho ni izquierdo, entonces se redefine como vacío. */
          Apunt = NULL;
        else
```

```
/* Si sólo tiene hijo izquierdo, el nodo eliminado se reemplaza con
éste.*/
          Apunt = ApAux3- > HijoIzq;
        else
        if (!ApAux3- > HijoIzq)
          /* Si sólo tiene hijo derecho, el nodo
          eliminado se reemplaza con éste. */
          Apunt = ApAux3- > HijoDer;
        else {
          /* Si tiene ambos hijos, entonces se reempla-
          za (en esta solución) por el nodo que está más a la derecha
del subárbol izquierdo. */
          ApAux1 = ApAux3- > HijoIzq;
          ApAux2 = ApAux3;
          while (ApAux1- > HijoDer) {
            ApAux2 = ApAux1;
            ApAux1 = ApAux1- > HijoDer;
          }
          ApAux3- > Info = ApAux1- > Info;
          if (ApAux3 == ApAux2)
            ApAux3- > HijoIzq = NULL;
          else
          if (!ApAux1- > HijoIzq)
            ApAux2- > HijoDer = NULL;
          else
            ApAux2- > HijoDer = ApAux1- > HijoIzq;
          ApAux3 = ApAux1;
        }
        delete(ApAux3);
      }
      Raiz = Apunt;
    }
```

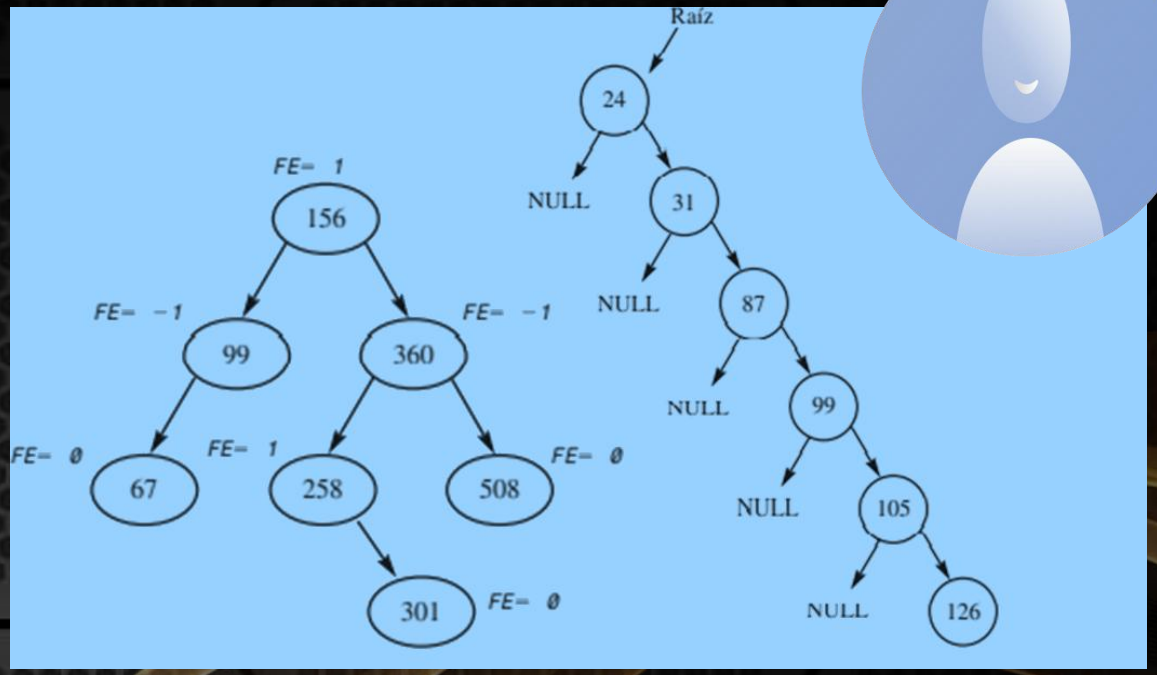


Arboles Balanceados

Reacomodo - Búsqueda – Inserción - Eliminación



Árboles Balanceados | Introducción



Un árbol balanceado es un árbol binario de búsqueda en el cual la diferencia entre la altura de su subárbol derecho y la altura de su subárbol izquierdo es menor o igual a 1. *De esta manera se controla el crecimiento del árbol y se garantiza mantener la eficiencia en la operación de búsqueda*. La diferencia entre las alturas de los subárboles se conoce como factor de equilibrio (FE), el cual se expresa como se muestra a continuación:

FE = altura hijo derecho – altura hijo izquierdo

La figura 7.12 muestra un árbol binario de búsqueda en el que cada nodo tiene un factor de equilibrio asociado. La raíz tiene un FE igual a 1 ya que el subárbol derecho tiene una altura de 3 y el izquierdo de 2. El nodo que almacena el valor 99 tiene un FE igual a –1 porque su subárbol derecho tiene altura 0 y el izquierdo 1. En cambio, el nodo que guarda el 508 tiene un FE igual a 0 porque sus dos subárboles tienen la misma altura. Al observar los FE de todos los nodos se puede afirmar que dicho árbol está balanceado, porque éstos son, en valor absoluto, menores o iguales a uno.

Considere que los elementos: 24 – 31 – 87 – 99 – 105 y 126 se almacenan, en ese orden, en un árbol binario de búsqueda. Luego de insertar los 6 valores, se obtiene un árbol como el de la figura 7.13. Si se tuviera que buscar un dato en este árbol se tendrían que hacer tantas comparaciones como nodos haya antes de llegar al deseado. En estructuras semejantes se pierden todas las ventajas que ofrecen los árboles. Para evitar que esto suceda surgen los árboles balanceados en los que se realizan balanceos o ajustes de los nodos luego de efectuar inserciones o eliminaciones que hayan provocado la pérdida del equilibrio

Árboles Balanceados | Reacomodo

. Para lograr que un árbol siga estando balanceado, luego de cada inserción o eliminación, se debe determinar si su factor de equilibrio es mayor a 1
En caso afirmativo, se deben reacomodar los nodos de tal manera que se vuelva a tener un valor menor o igual a 1. Este movimiento se denomina **rotación**. La rotación puede ser simple o compuesta dependiendo del número de nodos que participen.

La rotación simple se presenta cuando están involucrados dos hijos derechos (HD-HD) o dos hijos izquierdos (HI-HI), y afecta sólo las ligas de dos nodos. La figura 7.14 presenta gráficamente las dos variantes de este tipo de rotación.
Analizando el caso de la rotación HD-HD se observa que el nodo con información 24 tiene un FE igual a 2, lo cual indica que se ha perdido el equilibrio en su rama derecha. A su vez, en el nodo que guarda el 31 se tiene un FE igual a 1, con lo cual se puede determinar que la rotación debe involucrar también a la rama derecha de éste. Luego de efectuar la rotación, este nodo tiene como hijo izquierdo a su padre recupera
En el caso de la rotación HI-HI, se tiene el nodo que almacena el valor 99 con un FE igual a –2, lo cual indica que se ha perdido el equilibrio en su rama izquierda. Además, el nodo con información 85 tiene un FE igual a –1, con lo cual se puede determinar que la rotación también debe involucrar a la rama izquierda de éste. Luego de efectuar la rotación, este último nodo tiene como hijo derecho a su padre. Más adelante se explica cómo deben actualizarse los punteros a los nodos afectados durante la rotación simple

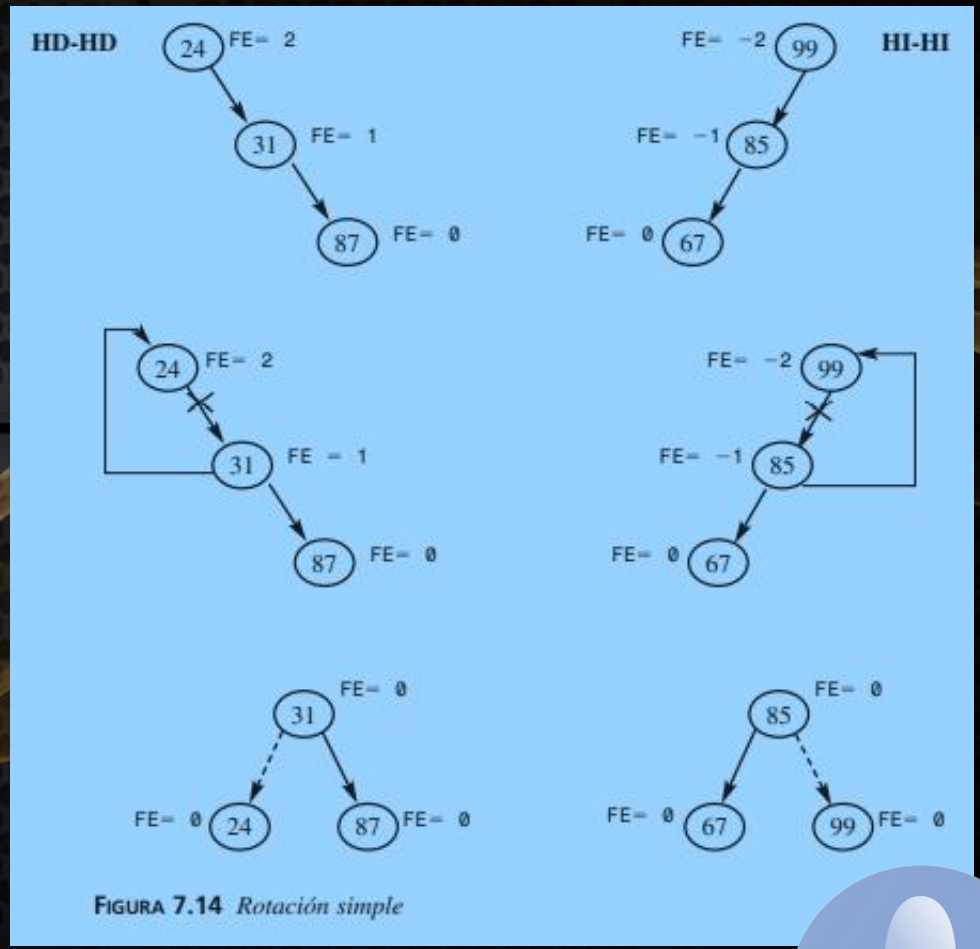


FIGURA 7.14 *Rotación simple*

Árboles Balanceados | Reacomodo

La rotación compuesta se presenta cuando están involucrados un hijo derecho y un hijo izquierdo (HD-HI) o un hijo izquierdo y uno derecho (HI-HD), y afecta las ligas de tres nodos. La figura 7.15 presenta gráficamente las dos variantes de este tipo de rotación

Analizando el caso de la rotación HD-HI se observa que el nodo con información 24 tiene un FE igual a 2, lo cual indica que se ha perdido el equilibrio en su rama derecha. A su vez, en el nodo que guarda el 31 se tiene un FE igual a –1, lo que implica que su rama izquierda tiene mayor altura que su rama derecha. Por tanto, la rotación que se debe aplicar es la de hijo derecho-hijo izquierdo. Luego de efectuar la rotación, el nodo 27 queda como padre del 24 (antes su abuelo) y del 31 (antes su padre). En el caso de la rotación HI-HD se observa que el nodo con información 99 tiene un FE igual a –2, lo cual indica que se ha perdido el equilibrio en su rama izquierda. Por otra parte, su hijo izquierdo (el nodo que guarda el 85) tiene un FE igual a 1, lo que implica que su rama derecha tiene mayor altura que su rama izquierda. Por lo tanto, la rotación que se debe aplicar es la de hijo izquierdo-hijo derecho. Luego de efectuar la rotación, el nodo que almacena el 90 queda como padre del 99 (antes su abuelo) y del 85 (antes su padre). En los árboles balanceados, cada nodo debe almacenar (además de los elementos ya vistos) su factor de equilibrio. Luego de insertar o eliminar un nodo se calculan los factores de equilibrio de todos los nodos involucrados en la operación y, dependiendo del valor resultante, se procede a realizar la rotación que corresponda y a hacer la reasignación.

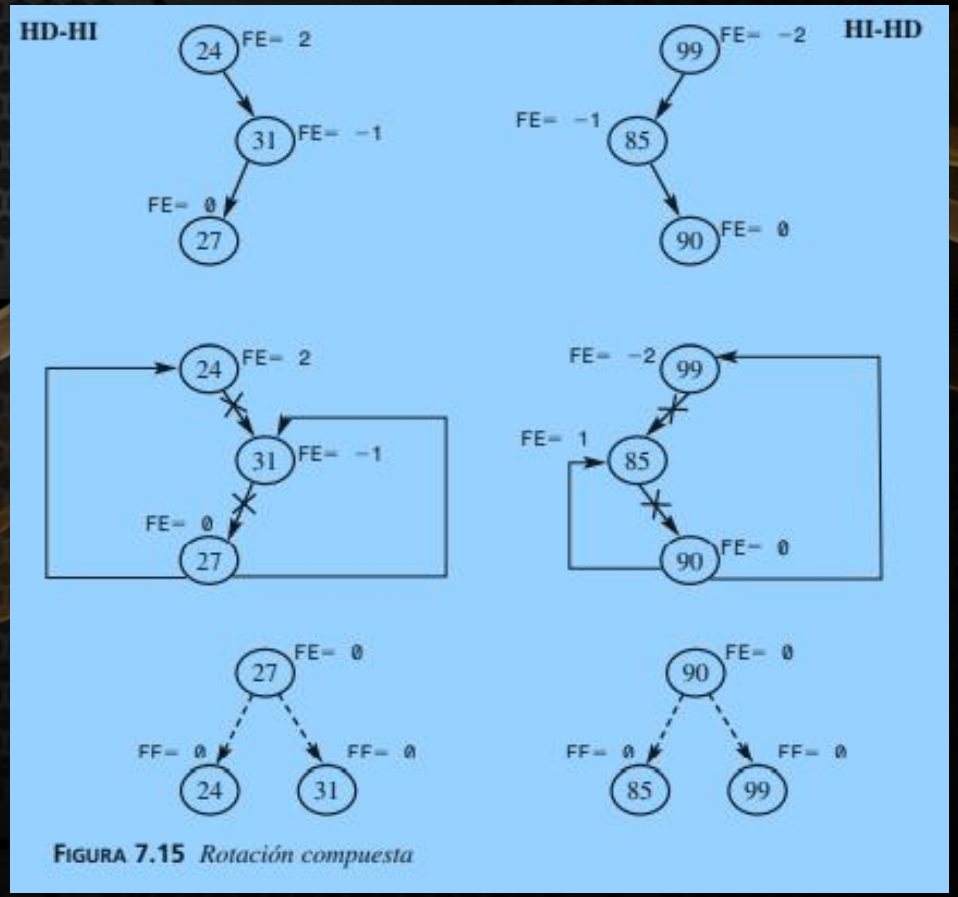


FIGURA 7.15 *Rotación compuesta*

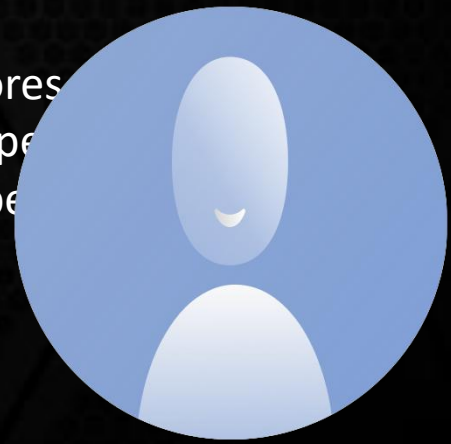
Árboles Balanceados | Inserción

Inserción en árboles balanceados

La inserción de un nuevo nodo se lleva a cabo atendiendo las características de los árboles binarios de búsqueda, pero teniendo en cuenta además la altura de los subárboles, de tal manera que no se viole lo mencionado sobre el factor de equilibrio. Los casos que pueden presentarse son:

1. La altura del subárbol derecho es igual a la altura del subárbol izquierdo, por lo tanto, sin importar dónde se realice la inserción, el equilibrio no se pierde.
2. La altura del subárbol derecho es mayor que la altura del subárbol izquierdo, por lo tanto, si la inserción no afecta la altura del subárbol derecho no se requiere rotación, en caso contrario sí.
3. La altura del subárbol izquierdo es mayor que la altura del subárbol derecho, por lo tanto, si la inserción no afecta la altura del subárbol izquierdo no se requiere rotación, en caso contrario sí.

Cualquiera que sea la situación, se procede a insertar el nuevo nodo y luego se actualizan los factores
equilibrio procediendo a la rotación de los nodos si correspondiera. El método para realizar esta ope
presenta a continuación. Las rotaciones simples y compuestas se escribieron como métodos indepe



Árboles Balanceados | Eliminación

Eliminación en árboles balanceados

La eliminación de un nodo se lleva a cabo atendiendo las características de los árboles binarios de búsqueda, pero teniendo en cuenta además la altura de los subárboles, de tal manera que no se viole lo mencionado sobre el factor de equilibrio.

Los casos que pueden presentarse son:

1. La altura del subárbol derecho es igual a la altura del subárbol izquierdo, por lo tanto, sin importar dónde se realice la eliminación, el equilibrio no se pierde.
2. La altura del subárbol derecho es mayor que la altura del subárbol izquierdo, por lo tanto, si la eliminación no afecta la altura del subárbol izquierdo no se requiere rotación, en caso contrario sí.
3. La altura del subárbol izquierdo es mayor que la altura del subárbol derecho, por lo tanto, si la eliminación no afecta la altura del subárbol derecho no se requiere rotación, en caso contrario sí.

Cualquiera que sea la situación, se elimina el nodo y luego se actualizan los factores de equilibrio de todos los nodos involucrados, procediendo a su rotación, si correspondiera. Este proceso termina cuando se llega a la raíz



Arboles-B

Búsqueda – Inserción - Eliminación

B Tree



Árboles-B

Las estructuras tipo árboles estudiadas hasta aquí son utilizadas para almacenar información en la memoria principal de la computadora. Sin embargo, en prácticamente todas las aplicaciones se requiere que los datos a procesar se guarden en dispositivos secundarios, de tal manera que permanezcan aún después de terminado el procesamiento. Además, el volumen de información manejado exige el uso de medios externos de almacenamiento. Por lo tanto, resulta necesario contar con estructuras que permitan organizar la información guardada en archivos. **Los árboles-B son una variante de los árboles balanceados y cubren esa necesidad**. En estas estructuras, a cada nodo se le conoce con el nombre de **página** y las páginas se guardan en algún dispositivo de almacenamiento secundario.

Las principales características de un árbol-B de grado n son:

- La página raíz almacena como mínimo 1 dato y como máximo 2n datos.
- La página raíz tiene como mínimo 2 descendientes.
- Las páginas intermedias y hojas almacenan entre n y 2n datos.
- Las páginas intermedias tienen entre (n+1) y (2n+1) páginas descendientes.
- Todas las páginas hojas tienen la misma altura.
- La información guardada en las páginas se encuentra ordenada.



Árboles B

La figura 7.16 presenta un ejemplo de un árbol-B, de grado 2. En la raíz se almacenan dos datos, lo que origina que tenga tres descendientes. La página hoja que está más a la izquierda guarda todos los datos que son menores al primer dato (105) de la página raíz, la segunda hoja contiene los datos mayores a 105 y menores a 320, mientras que la tercera hoja almacena los datos mayores al segundo dato de la página raíz (320). Cada una de las páginas hojas tiene entre 2 y 4 elementos. Si no fueran hojas, tendrían: la primera 3, la segunda 5 y la tercera 4 páginas descendientes respectivamente

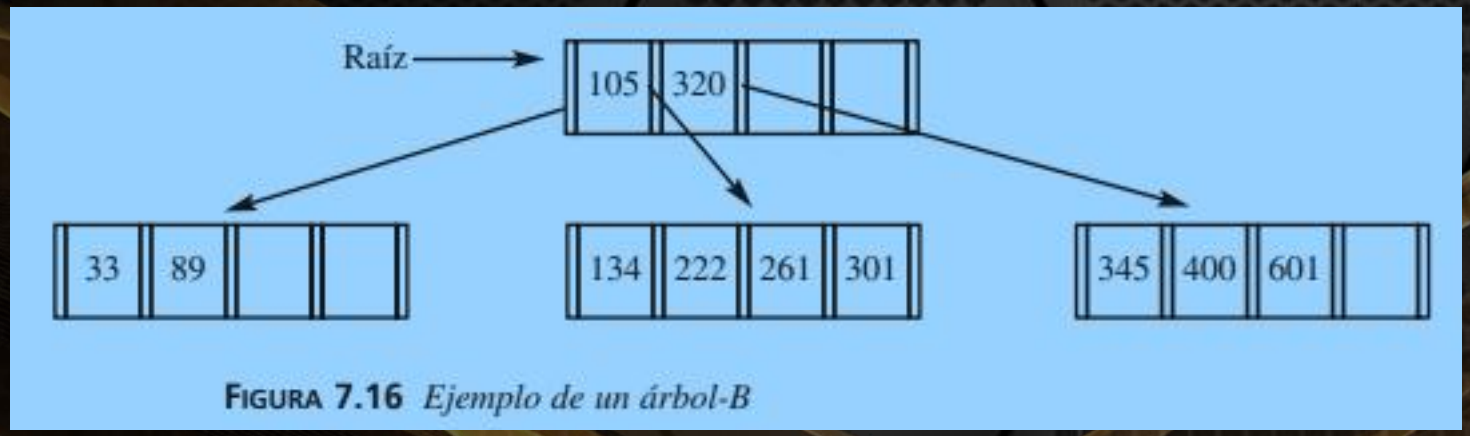


FIGURA 7.16 *Ejemplo de un árbol-B*

Las operaciones que pueden realizarse sobre la información almacenada en árboles-B son: búsqueda, inserción y elimi
Estas operaciones están implementadas en herramientas diseñadas especialmente para el manejo de archivos, por l
generalmente no se requiere su programación. En esta parte de la unidad se analizan para entenderlas y cuando ha
algún manejador de archivos sepa qué está pasando internamente con los datos



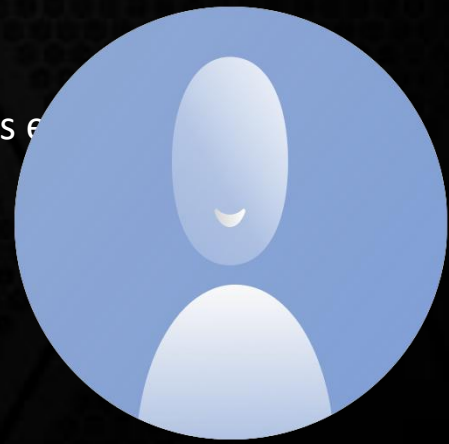
Árboles B - Búsqueda en árboles-B

La operación de **búsqueda** es similar a la estudiada en los árboles binarios de búsqueda, por ser los árboles-B una generalización de los primeros. En este tipo de árboles se recuperan del medio secundario de almacenamiento páginas completas de información y se procede a buscar en ellas el dato deseado. Si se encuentra, termina la búsqueda; en caso contrario se procede con la página que corresponda según el dato con el cual se compara, ya sea menor o mayor. Si se requiere recuperar una nueva página pero no existe, entonces se tiene un caso de fracaso.

Los pasos para llevar a cabo esta operación son los siguientes:

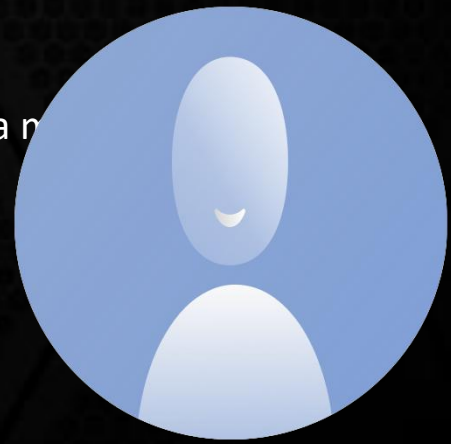
1. Se recupera una página (la primera vez es la página raíz) y se la lleva a memoria.
2. Se evalúa si la página está vacía. 2.1. Si la respuesta es afirmativa, entonces la búsqueda termina con fracaso.
   2.2. Si la respuesta es negativa, entonces ir al paso 3.
3. Se compara el dato buscado con cada elemento almacenado en la página.
   3.1. Si es igual, la búsqueda termina con éxito.
   3.2. Si es menor se toma la dirección de sus descendientes por el lado izquierdo y se regresa al paso 1.
   3.3. Si es mayor, se avanza al siguiente dato de la misma página. 3.3.1.

   Si es el último, se toma la dirección de sus descendientes por el lado derecho y se regresa al paso 1. 3.3.2. Si no es e
   regresa al paso 3.



Árboles B - Búsqueda en árboles-B

1 Se recupera la página con los datos: 105 – 320
2 Se evalúa si está vacía. En este caso la respuesta es negativa.
3 Se compara el dato buscado (319) con el primer elemento de la página (105). Es mayor y hay más elementos en la misma página, entonces se avanza al siguiente valor (320).
4 Se compara el dato buscado (319) con el valor 320. Es menor, entonces se toma la dirección de la página que está a la izquierda del 320.
5 Se recupera la página con los datos: 134 – 222 – 261 – 301.
6 Se evalúa si está vacía. En este caso la respuesta es negativa.
7 Se compara el dato buscado (319) con el primer elemento de la página (134). Es mayor y hay más elementos en la misma página, entonces se avanza al siguiente valor (222).
8 Se compara el dato buscado (319) con el valor 222. Es mayor y hay más elementos en la misma página, entonces se avanza al siguiente valor (261).
9 Se compara el dato buscado (319) con el 261. Es mayor y hay más elementos en la misma página, entonces se avanza al siguiente valor (301).
10 Se compara el dato buscado (319) con el valor 301. Es mayor y ya no hay más elementos en la misma página, entonces se toma la dirección de la página que está a la derecha del 301.
11 Se recupera la página con los datos: 310 – 319.
12 Se evalúa si está vacía. En este caso la respuesta es negativa.
13 Se compara el dato buscado (319) con el primer elemento de la página (310). Es mayor y hay más elementos en la p
entonces se avanza al siguiente valor (319).
14 Se compara el dato buscado (319) con el valor 319. Es igual, por lo tanto la búsqueda termina con éxito



Árboles B



En la figura 7.17 las líneas punteadas indican las páginas que se van visitando hasta llegar al dato buscado. Por su parte, las páginas sombreadas son las que se recuperan para hacer la comparación del elemento buscado con los elementos almacenados en el árbol.

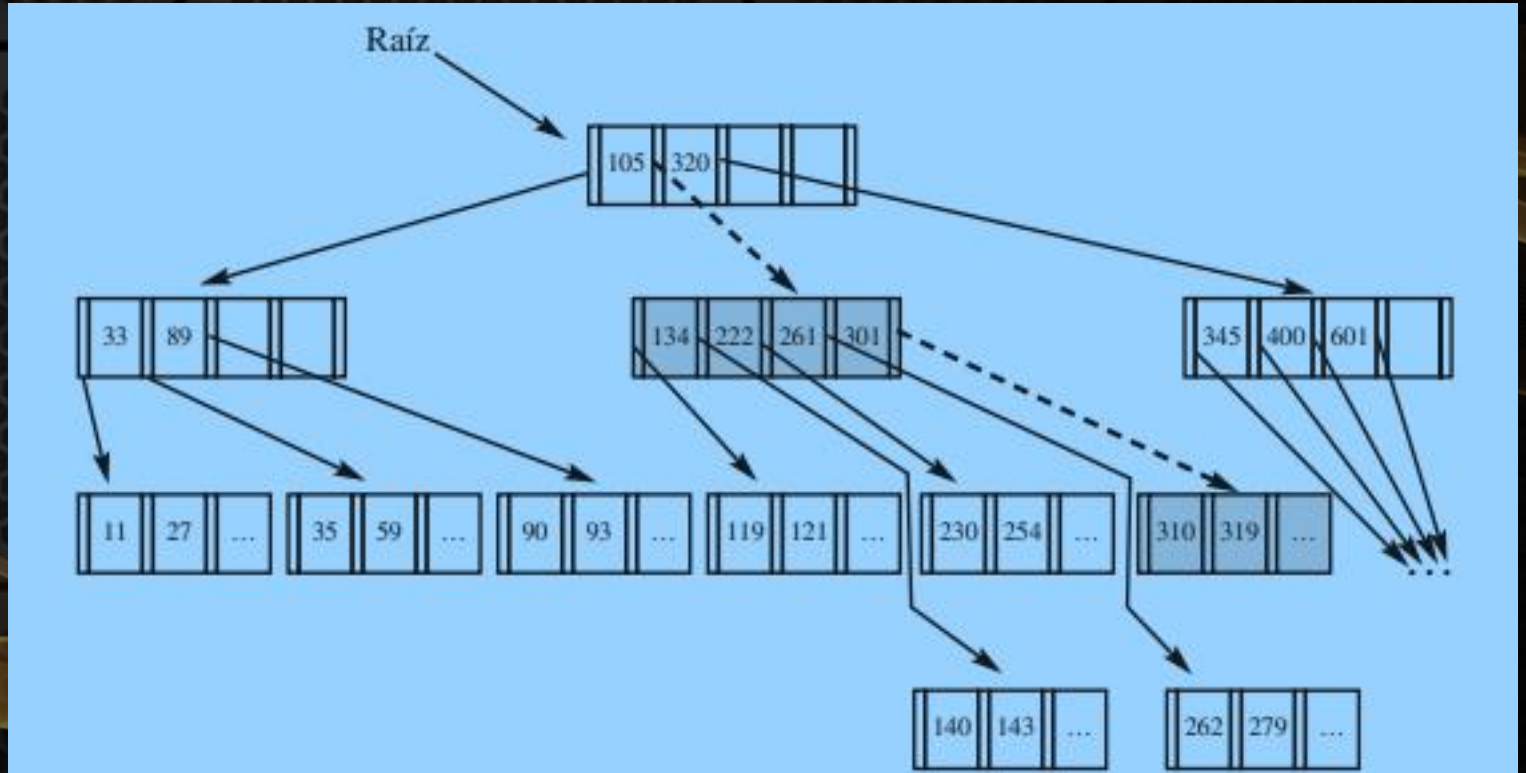


FIGURA 7.17 *Búsqueda en árboles-B*

Árboles B - Inserción en árboles-B

Inserción en árboles-B

La operación de **inserción** se caracteriza porque los nuevos elementos siempre se guardan a nivel de las hojas, y puede originar la reestructuración del árbol incluso hasta la raíz provocando esto último que la altura aumente en uno. Los pasos para llevar a cabo esta operación son los siguientes:

1. Se recupera una página (la primera vez es la página raíz).
2. 2. Se evalúa si es una página hoja.

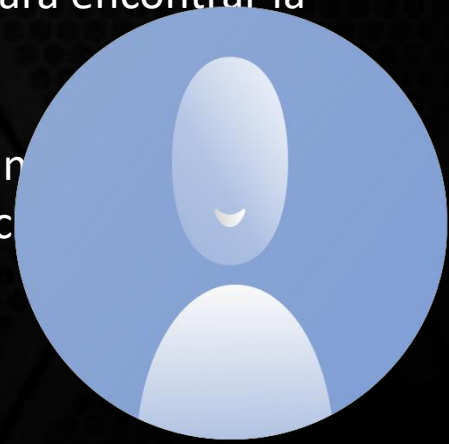
   2.1. Si la respuesta es afirmativa, se evalúa si la cantidad de elementos almacenados en ella es menor a 2n, siendo n el grado del árbol.

   2.1.1. Si la respuesta es afirmativa, entonces se procede a insertar el nuevo valor en el lugar correspondiente.

   2.1.2. Si la respuesta es negativa, entonces se divide la página en dos y los (2n + 1) datos se distribuyen entre las páginas resultantes (n en cada página) y el valor del medio sube a la página padre. Si la página padre no tuviera espacio para este valor, entonces se procede de la misma manera, se divide en dos y el elemento del medio sube a la siguiente página, pudiendo repetirse este proceso hasta llegar a la raíz, en cuyo caso se aumenta la altura del árbol.

   2.2. Si no es una hoja, se compara el elemento a insertar con cada uno de los valores almacenados para encontrar la página descendiente donde proseguir la búsqueda. Se sigue con el paso 1.

   Considere el árbol-B, de grado 2, de la figura 7.18 en el cual se quiere insertar el valor 60. La línea pun
   página que se recupera y donde se agrega el 60. En la tabla 7.5 se presenta la secuencia de pasos nec
   realizar esta operación



Árboles B - Inserción en árboles-B

Operación de inserción en un árbol-B

1 Se recupera la página con los valores: 105 – 320.
2 Se evalúa si es una página hoja. No lo es.
3 Se compara el dato a insertar (60) con el valor 105. Es menor, entonces se toma la dirección de la página que está a la izquierda del 105.
4 Se recupera la página con los valores: 33 – 89.
5 Se evalúa si es una página hoja. Sí lo es.
6 Se evalúa si el total de elementos almacenados (2) es menor a 2n. Sí lo es.
7 Se inserta el valor 60 entre el 33 y 89 de tal manera que no altere el orden

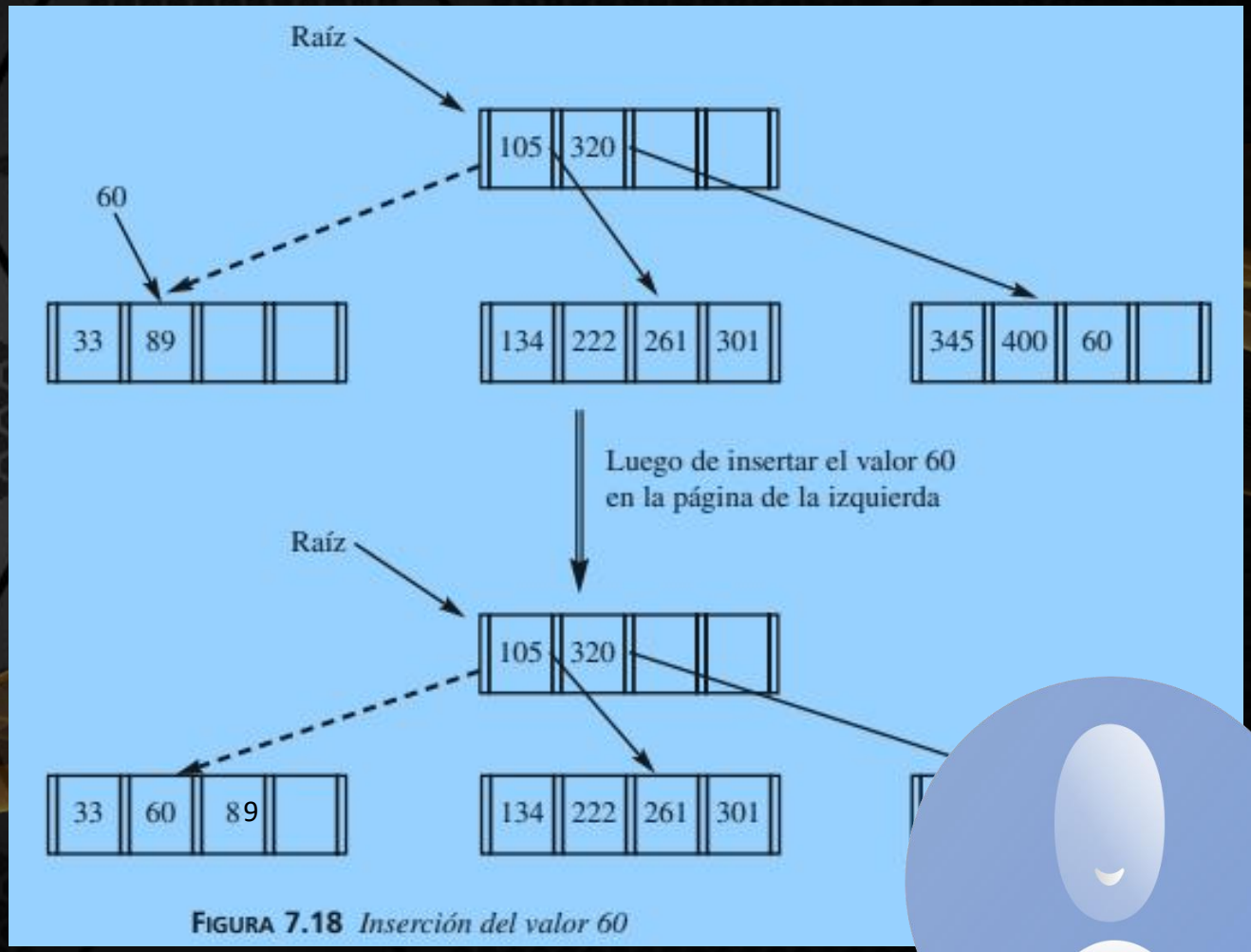


FIGURA 7.18 *Inserción del valor 60*

En el ejemplo anterior no hubo cambios en la estructura del árbol. Considere ahora el árbol-B, de grado 2, de la figura 7.19 en el cual se quiere insertar el valor 120. En la tabla 7.6 se presenta la secuencia de operaciones realizadas al aplicar el algoritmo visto.

Árboles B - Inserción en árboles-B

En el ejemplo anterior no hubo cambios en la estructura del árbol. Considere ahora el árbol-B, de grado 2, de la figura 7.19 en el cual se quiere insertar el valor 120. En la tabla 7.6 se

1 Se recupera la página con los valores: 105 – 320.
2 Se evalúa si es página hoja. No lo es.
3 Se compara el dato a insertar (120) con el valor 105. Es mayor y hay más elementos en la misma página, entonces se avanza al siguiente valor (320).
4 Se compara el dato a insertar (120) con el valor 320. Es menor, entonces se toma la dirección de la página que está a la izquierda del 320.
5 Se recupera la página con los valores: 134 – 222 – 261 – 301.
6 Se evalúa si es página hoja. Sí lo es.
7 Se evalúa si el total de elementos almacenados (4) es menor a 2n. No lo es.
8 Se divide la página en dos y el valor del medio (222) sube a la página padre, en la cual hay espacio. Los demás valores se distribuyen entre las dos nuevas páginas.

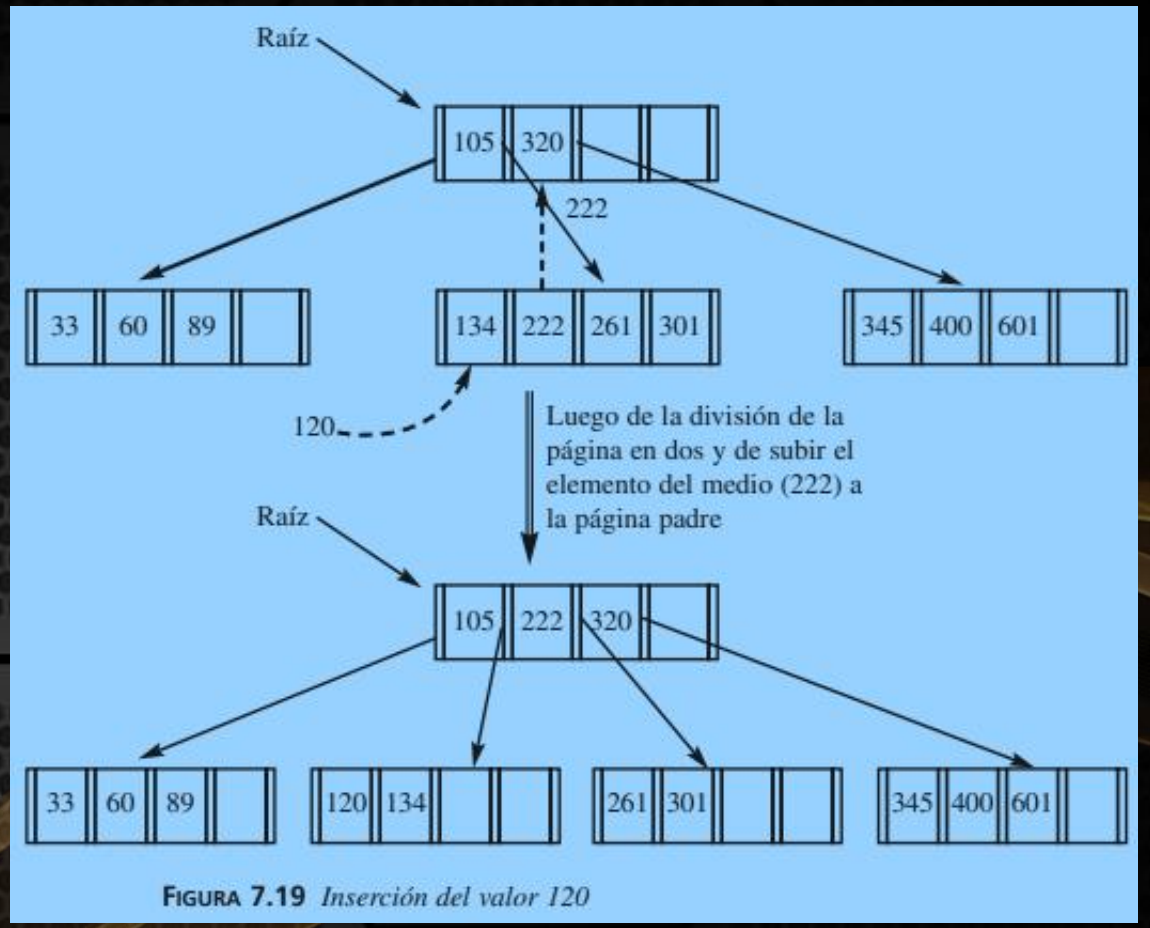


FIGURA 7.19 *Inserción del valor 120*

Como se puede apreciar en la figura 7.19, luego
valor 120 se modifica la estructura del árbol-B
que debía insertarse el nuevo dato estaba co
se dividió en dos y subió el dato central a la
Finalmente la página padre quedó con tres e
tanto con cuatro descendientes



Árboles B

A continuación se presenta un ejemplo en el cual la estructura del árbol se modifica en cuanto al número de páginas y a la altura. En el árbol-B, de grado 2, de la figura 7.20 se quiere insertar el valor 850. La tabla 7.7 muestra las operaciones que se realizaron al aplicar el algoritmo de inserción.

1 Se recupera la página con los valores: 105 – 320 – 505 – 720
2 Se evalúa si es página hoja. No lo es.
3 Se compara el dato a insertar (850) con el valor 105. Es mayor y hay más elementos en la misma página, entonces se avanza al siguiente valor (320).
4 Se compara el dato a insertar (850) con el valor 320. Es mayor y hay más elementos en la misma página, entonces se avanza al siguiente valor (505). 5 Se compara el dato a insertar (850) con el valor 50
5. Es mayor y hay más elementos en la misma página, entonces se avanza al siguiente valor (720).
6 Se compara el dato a insertar (850) con el valor 720. Es mayor y ya no hay más elementos en la misma página, entonces se toma la dirección de la página que está a la derecha del 720.
7 Se recupera la página con los valores: 765 – 800 – 801 – 976.
8 Se evalúa si es página hoja. Sí lo es.
9 Se evalúa si el total de elementos almacenados (4) es menor a 2n. No lo es.
10 Se divide la página en dos y el valor del medio (801) sube a la página padre. Los demás valores se distribuyen entre las dos nuevas páginas.
11 En la página padre no hay espacio, el número de elementos almacenados es igual a 2n. Por lo tanto, se debe dividir en dos y el dato del medio (505) debe guardarse en una página que será la nueva raíz del árbol. La altura del árbol crece en una unidad

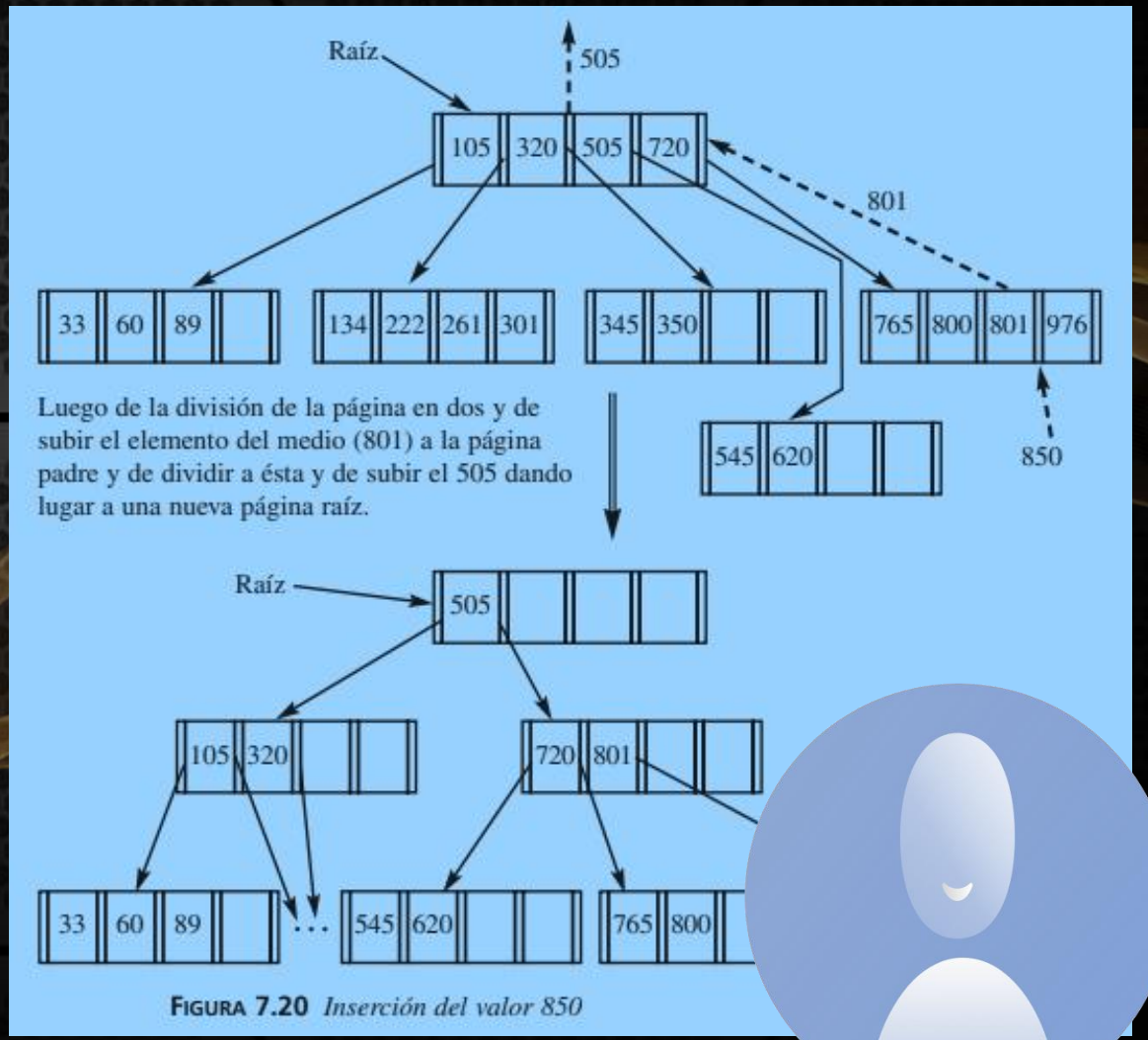
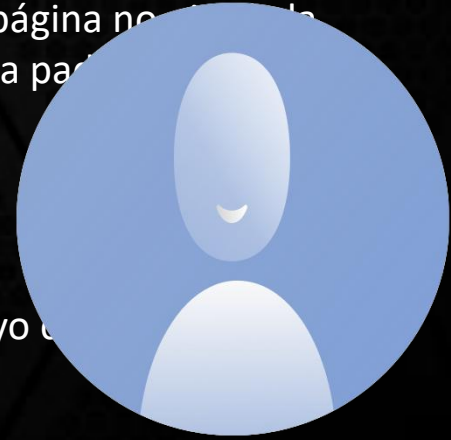


FIGURA 7.20 *Inserción del valor 850*

Árboles B - Eliminación en árboles-B

La operación de **eliminación** consiste en quitar un elemento del árbol-B cuidando que mantenga las propiedades vistas. Es decir, el número de datos en cada página debe ser mayor o igual a n y menor o igual a 2n. Los pasos para llevar a cabo esta operación son los siguientes:

1. Se recupera una página (la primera vez es la página raíz) y se la lleva a memoria.
2. Se evalúa si la página está vacía.
   2.1. Si la respuesta es afirmativa, entonces la operación de eliminación termina con fracaso.
   2.2. Si la respuesta es negativa, entonces ir al paso 3.
3. Se compara el dato a eliminar con cada elemento almacenado en la página.
   3.1. Si es igual, entonces se elimina y se procede de la siguiente manera:
      3.1.1. Si el dato estaba en una página hoja y el número de elementos de ésta sigue siendo un valor comprendido entre n y 2n, entonces la operación de eliminación termina.
      3.1.2. Si el dato estaba en una página y el número de elementos de ésta queda menor que n, entonces se debe bajar el dato más cercano de la página padre y sustituirlo por el que se encuentre más a la izquierda del subárbol derecho o por el que se encuentre más a la derecha del subárbol izquierdo, siempre que esta página no pierda la condición y se fusionan.
      3.1.3. Si el dato estaba en la página raíz o en una página intermedia, entonces se debe sustituir por el que se encuentre más a la izquierda del subárbol derecho o por el que se encuentre más a la derecha del subárbol izquierdo, siempre que esta página no [illegible] condición. Si es así, termina la eliminación con éxito. En caso contrario, se debe bajar el dato más cercano de la página pa[illegible] las páginas que son hijas de éste.
   3.2. Si es menor se toma la dirección de sus descendientes por el lado izquierdo y se regresa al paso 1.
   3.3. Si es mayor, se avanza al siguiente dato de la misma página.
      3.3.1. Si es el último, se toma la dirección de sus descendientes por el lado derecho y se regresa al paso 1.
      3.3.2. Si no es el último, se regresa al paso 3. El proceso de fusión de páginas puede llegar hasta la raíz, en cuyo [illegible] árbol disminuye en uno.



Árboles B

Analice el siguiente ejemplo. Se tiene un árbol-B, de grado 2 (ver figura 7.21) en el cual se quiere eliminar el valor 222. Aplicando el algoritmo dado, se realizan las operaciones que aparecen en la tabla 7.8

1 Se recupera la página con los valores: 105 – 320.
2 Se evalúa si la página está vacía. No lo está.
3 Se compara el dato a eliminar (222) con el valor 105. Es mayor y hay más elementos en la misma página, entonces se avanza al siguiente valor (320).
4 Se compara el dato a eliminar (222) con el valor 320. Es menor, entonces se toma la dirección de la página que está a la izquierda del 320.
5 Se recupera la página con los valores: 134 – 222 – 261 – 301.
6 Se compara el dato a eliminar (222) con el valor 134. Es mayor y hay más elementos en la misma página, entonces se avanza al siguiente valor (222).
7 Se compara el dato a eliminar (222) con el valor 222. Es igual, entonces se elimina.
8 Se evalúa si el dato eliminado estaba en una página hoja. Sí lo estaba.
9 Se evalúa si el número de elementos (3) es n y 2n. Sí lo es. La operación termina con éxito.

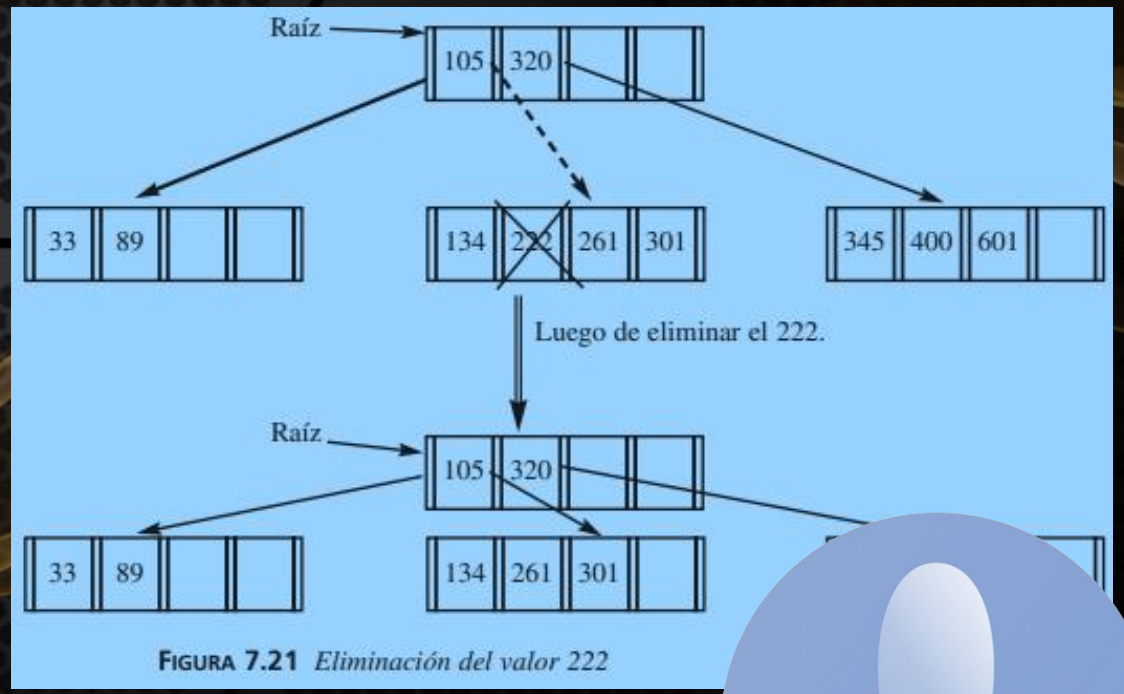


FIGURA 7.21 *Eliminación del valor 222*

Árboles B

En el ejemplo anterior se presentó el caso más simple de eliminación. El dato buscado estaba en una página hoja que a su vez tenía más de n elementos. Observe ahora el siguiente caso. En la figura 7.22 se tiene un árbol-B, de grado 2, del cual se quiere eliminar el valor 89. Aplicando el algoritmo dado, se llevan a cabo las operaciones mostradas en la tabla 7.9.

1 Se recupera la página con los valores: 105 – 320.
2 Se evalúa si la página está vacía. No lo está.
3 Se compara el dato a eliminar (89) con el valor 105. Es menor, entonces se toma la dirección de la página que está a la izquierda del 105.
4 Se recupera la página con los valores: 33 – 89.
5 Se compara el dato a eliminar (89) con el valor 33. Es mayor y hay más elementos en la misma página, entonces se avanza al siguiente valor (89).
6 Se compara el dato a eliminar (89) con el valor 89. Es igual, entonces se elimina.
7 Se evalúa si el dato eliminado estaba en una página hoja. Sí lo estaba.
8 Se evalúa si el número de elementos (1) es n y 2n. No lo es, entonces se debe bajar el dato más cercano de la página padre (105) y sustituir a éste por el que se encuentre más a la izquierda del subárbol derecho (134).
9 Se evalúa si el número de elementos de la página del subárbol derecho (de la cual se quitó el 134) cumple con la condición. En este caso (2) es n y 2n. La operación termina con éxito

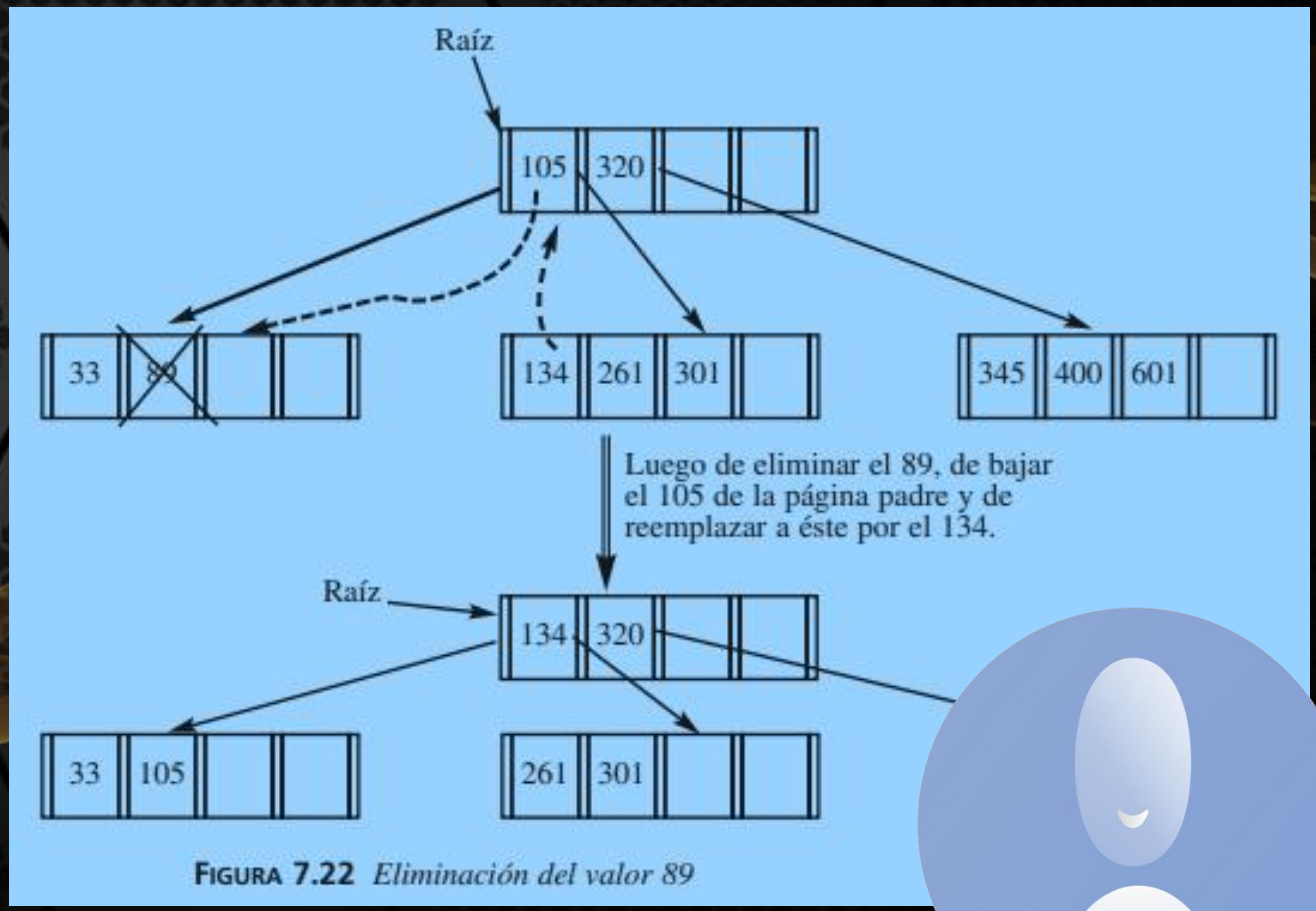


FIGURA 7.22 *Eliminación del valor 89*

Árboles B

Por último, analice el siguiente ejemplo. En el árbol-B, de grado 2, de la figura 7.23 se quiere eliminar el valor 48. En este caso la altura del árbol, luego de efectuar todas las reestructuraciones que corresponden, disminuye en uno La tabla 7.10 presenta las operaciones que se realizan al aplicar el algoritmo.

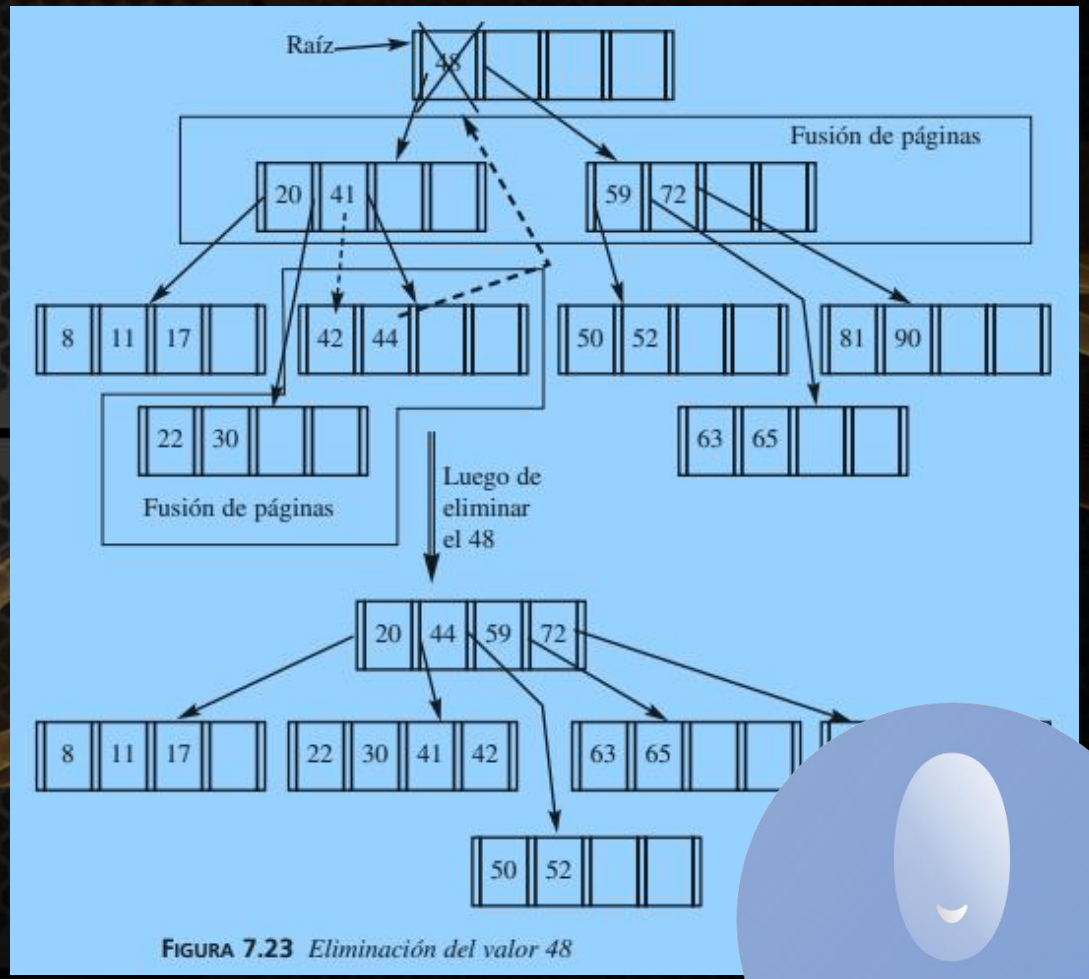


FIGURA 7.23 *Eliminación del valor 48*

1 Se recupera la página con los valores: 48.
2 Se evalúa si la página está vacía. No lo está.
3 Se compara el dato a eliminar (48) con el valor 48. Es igual, entonces se elimina.
4 Se evalúa si el dato eliminado estaba en una página hoja. No lo estaba.
5 Se debe sustituir el elemento eliminado por el que se encuentre más a la derecha del subárbol izquierdo (44)
6 Se evalúa si el número de elementos (1) de esa página es n y 2n. No lo es, entonces se debe bajar el dato más cercano (41) de la página padre y fusionar las páginas que son hijas de éste (22 – 30 – 41 – 42).
7 Se evalúa si el número de elementos de la página de la que se bajó el valor 41 es n y 2n. Es (1), por lo tanto se debe bajar el dato más cercano (44) de la página padre y fusionar las páginas que son hijas de éste (20 – 44 – 59 – 72). La fusión afectó la raíz, disminuyendo en 1 la altura del árbol. La operación termina con éxito

Arboles B+

Búsqueda – Inserción – Eliminación

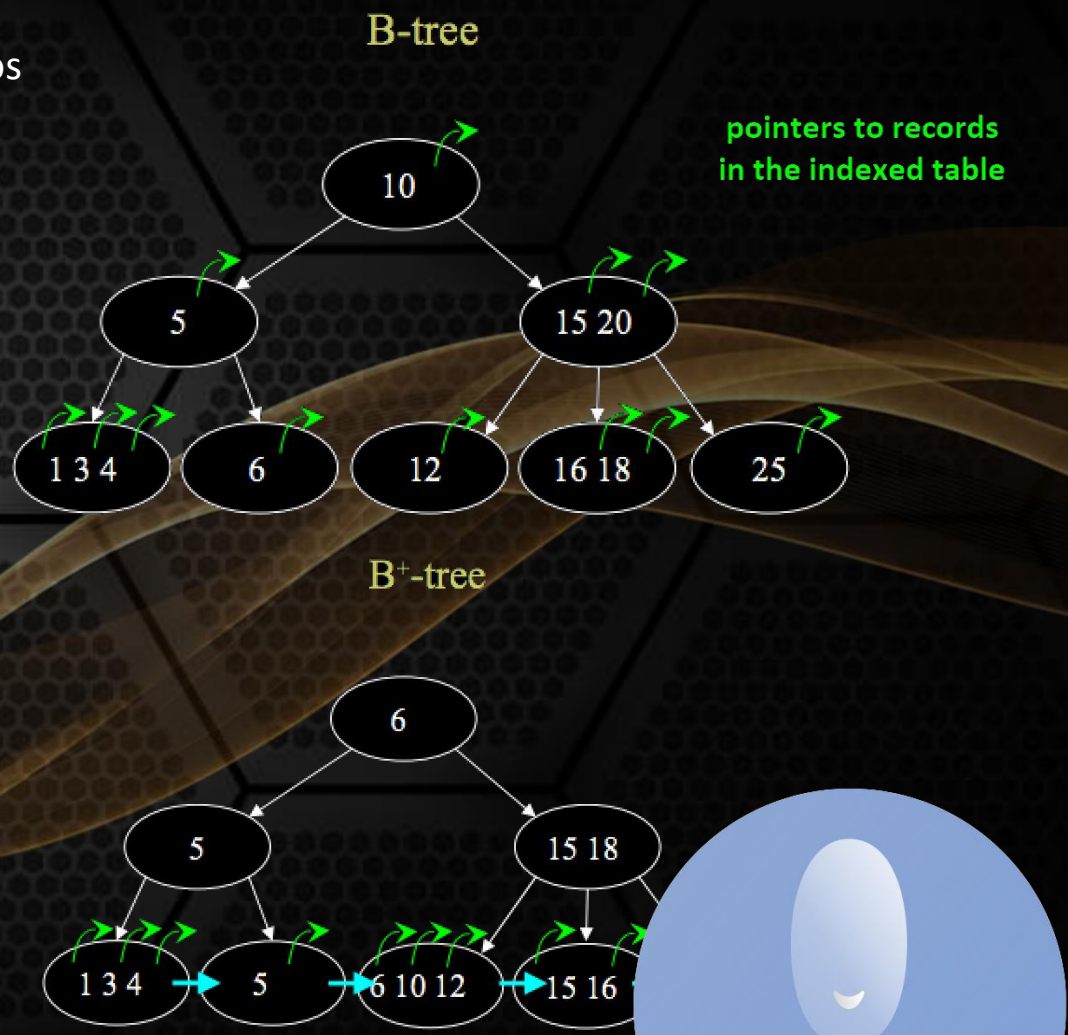
B+ Tree



Árboles-B+

Los árboles-B+ son una variante de los árboles-B, diferenciándose de estos últimos por el hecho de que toda la información se encuentra almacenada en las hojas. En la raíz y en los nodos intermedios se guardan solamente las claves o índices que permiten llegar a un cierto dato.
Las principales características de un árbol-B+ de grado n son:

- La página raíz almacena como mínimo 1 dato y como máximo 2n datos.
- La página raíz tiene como mínimo 2 descendientes.
- Las páginas intermedias y hojas almacenan entre n y 2n datos.
- Las páginas intermedias tienen entre (n+1) y (2n+1) páginas descendientes.
- Todas las páginas hojas tienen la misma altura.
- La información se encuentra ordenada.
- Toda la información se encuentra en las páginas hojas, por lo que la clave guardada en la raíz o páginas intermedias se duplica.
- La información guardada en la raíz o en páginas intermedias cumple la función de índices que facilitan el acceso a un cierto dato



Árboles-B+

a figura 7.24 presenta un ejemplo de un árbol-B+, de grado 2. En la raíz se almacenan valores que funcionan como índices para llegar a los datos que están en las hojas (es de suponer que el árbol almacena datos más complejos que simples números enteros).

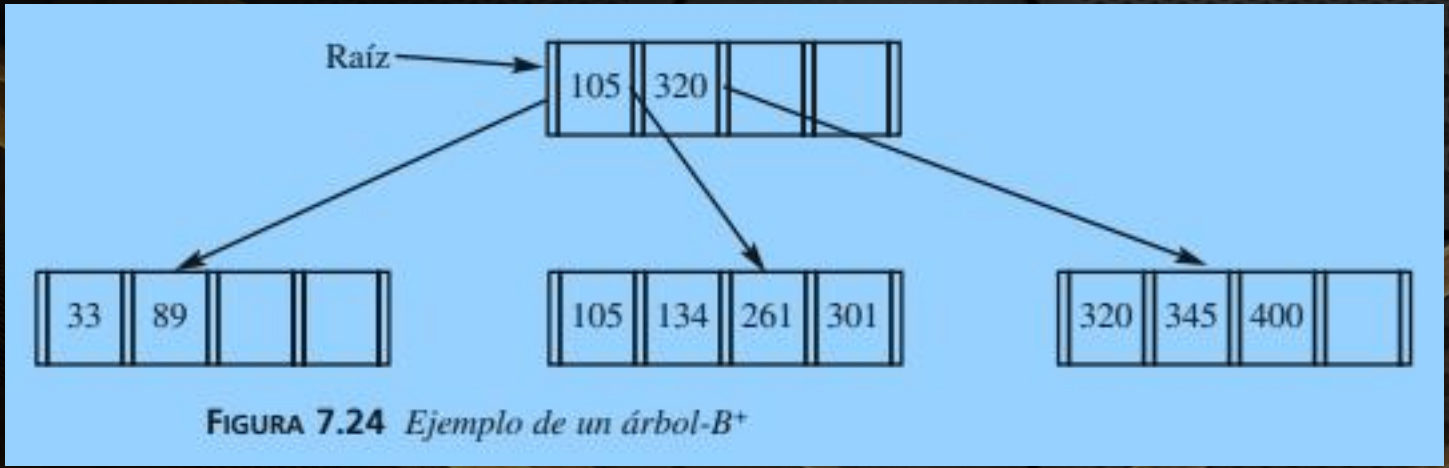


FIGURA 7.24 *Ejemplo de un árbol-B+*

Como en el caso de los árboles-B, se explican para que pueda entenderlas y cuando haga uso de algún manejador de archivos sepa qué está pasando internamente con los datos.

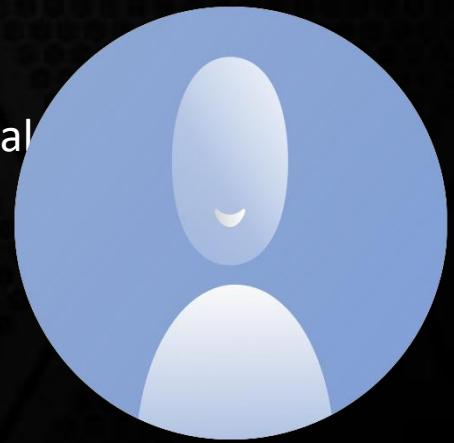
Las operaciones que pueden realizarse sobre la información almacenada en árboles-B+ son: búsqueda, inserción y eliminación. Estas operaciones están implementadas en herramientas diseñadas especialmente para el manejo de archivos, por lo que generalmente no se requiere su programación en las aplicaciones.



Árboles-B+ Busqueda

Búsqueda en árboles-B+

La operación de búsqueda es similar a la estudiada en los árboles-B. La diferencia es que en estos árboles la búsqueda termina siempre en las páginas hojas (donde está la información completa).
Los pasos para llevar a cabo esta operación son los siguientes:
1. Se recupera una página (la primera vez es la página raíz) y se la lleva a memoria.
2. Se evalúa si la página está vacía.
    2.1. Si la respuesta es afirmativa, entonces la búsqueda termina con fracaso.
    2.2. Si la respuesta es negativa, entonces ir al paso 3.
3. Se compara el dato buscado con cada elemento almacenado en la página.
    3.1. Si es igual, entonces se evalúa si es una página hoja.
        3.1.1. Si la respuesta es afirmativa, entonces la búsqueda termina con éxito.
        3.1.2. Si la respuesta es negativa, entonces se debe recuperar la página descendiente por el lado derecho y se regresa al paso 1.
3.2. Si es menor se toma la dirección de sus descendientes por el lado izquierdo y se regresa al paso 1.
3.3. Si es mayor, se avanza al siguiente dato de la misma página.
        3.3.1. Si es el último, se toma la dirección de sus descendientes por el lado derecho y se regresa al
        3.3.2. Si no es el último, se regresa al paso 3.



Árboles-B+ Busqueda

1 Se recupera la página con los datos: 105 – 320.
2 Se evalúa si está vacía. En este caso la respuesta es negativa.
3 Se compara el dato buscado (320) con el primer elemento de la página (105). Es mayor y hay más elementos en la misma página, entonces se avanza al siguiente valor (320).
4 Se compara el dato buscado (320) con el valor 320. Es igual.
5 Se evalúa si la página donde fue encontrado el 320 es una página hoja. No lo es. Se debe revisar la página descendiente por el lado derecho.
6 Se recupera la página con los datos: 320 – 345 – 400.
7 Se compara el dato buscado (320) con el primer elemento de la página (320). Es igual.
8 Se evalúa si la página donde fue encontrado el 320 es una página hoja. Sí lo es. La búsqueda termina con éxito



Árboles-B+ Inserción

Inserción en árboles-B+

1. Se recupera una página (la primera vez es la página raíz).
   2. Se evalúa si es una página hoja.
      2.1. Si la respuesta es afirmativa, se evalúa si la cantidad de elementos almacenados en ella es menor a 2n.
         2.1.1. Si la respuesta es afirmativa, entonces se procede a insertar el nuevo valor en el lugar correspondiente.
         2.1.2. Si la respuesta es negativa, se divide la página en dos y los (2n + 1) datos se distribuyen entre las páginas resultantes y una copia del valor del medio (o de una clave del mismo) sube a la páginapadre. Si la página padre no tuviera espacio para este valor, entonces se procede de la misma manera, se divide en dos y el elemento del medio sube a la siguiente página, pudiendo repetirse este proceso hasta llegar a la raíz, en cuyo caso se aumenta la altura del árbol.
      2.2. Si no es una hoja, se compara el elemento a insertar con cada uno de los valores almacenados para encontrar la página descendiente donde proseguir la búsqueda. Se regresa al paso 1.



Árboles-B+ Inserción



Considere el siguiente ejemplo. En el árbol-B+, de grado 2, de la figura 7.25 se quiere insertar el valor 287. Aplicando el algoritmo dado, se realizan las operaciones que se muestran en la tabla 7.12.

1 Se recupera la página con los valores: 105 – 320.
2 Se evalúa si es una página hoja. No lo es.
3 Se compara el dato a insertar (287) con el valor 105. Es mayor y hay más elementos en la misma página, entonces se avanza al siguiente valor (320).
4 Se compara el dato a insertar (287) con el valor 320. Es menor, entonces se toma la dirección de la página que está a la izquierda del 320.
5 Se recupera la página con los valores: 105 – 222 – 261 – 301.
6 Se evalúa si es una página hoja. Sí lo es.
7 Se evalúa si el total de elementos almacenados (4) es menor a 2n. No lo es.
8 Se divide la página en dos y los elementos se distribuyen entre ellas, subiendo una copia del valor del medio (261) a la página padre.
9 Se evalúa (antes de la inserción) si el número de elementos en la página padre (2) es menor a 2n. Sí lo es. La operación termina con éxito.

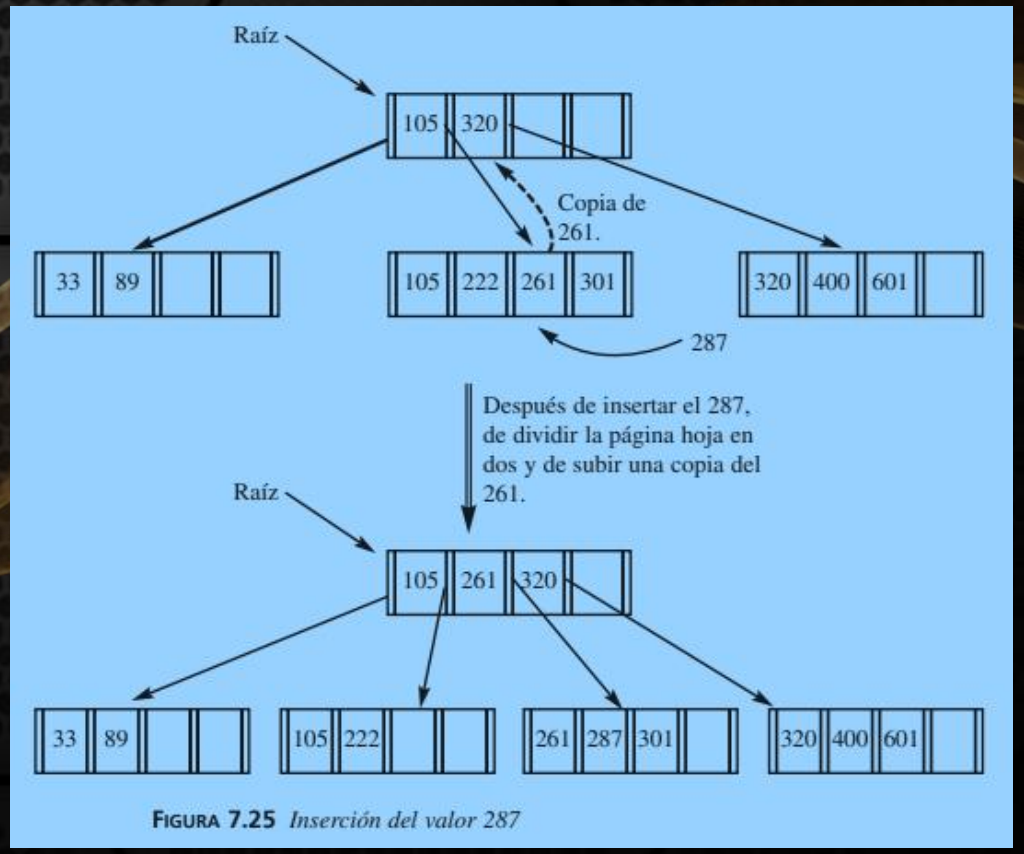


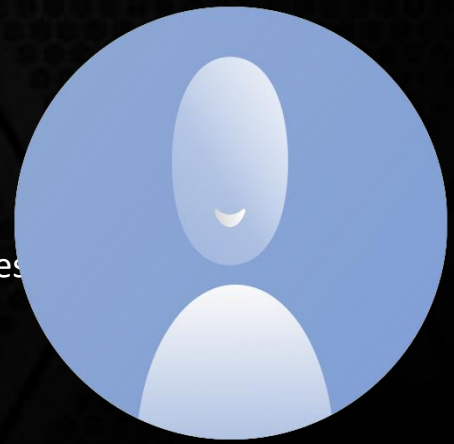
FIGURA 7.25 *Inserción del valor 287*

Árboles-B+ - Eliminación en árboles-B+

La operación de eliminación consiste en quitar un elemento del árbol-B+ cuidando que mantenga las propiedades vistas. Es decir, el número de datos en cada página debe ser mayor o igual a n y menor o igual a 2n. Como los datos siempre están en las páginas hojas, cuando se encuentran se quitan (sólo de la hoja, sin importar si además están en un nodo intermedio o raíz) y sólo se reestructura el árbol si la página quedó con menos de n elementos. Los pasos para llevar a cabo esta operación son los siguientes:

1. Se recupera una página (la primera vez es la página raíz) y se la lleva a memoria.
2. Se evalúa si la página está vacía.
    2.1. Si la respuesta es afirmativa, entonces la operación de eliminación termina con fracaso.
    2.2. Si la respuesta es negativa, entonces ir al paso 3.
3. Se compara el dato a eliminar con cada elemento almacenado en la página.
    3.1. Si es igual, se evalúa si está en una página hoja.
        3.1.1. Si la respuesta es negativa entonces se toma la dirección de sus descendientes por el lado derecho y se regresa al paso 1.
        3.1.2. Si la respuesta es afirmativa, entonces se elimina y se evalúa si el número de elementos que quedó sigue siendo mayor o igual a n.
            3.1.2.1 Si la respuesta es afirmativa, entonces la operación termina con éxito. Las páginas intermedias no se modifican aunque almacenen una copia del elemento eliminado.
            3.1.2.2 Si la respuesta es negativa, entonces se debe bajar el dato más cercano de la página padre y sustituir a éste por el que se encuentre más a la izquierda del subárbol derecho o por el que se encuentre más a la derecha del subárbol izquierdo, siempre que esta página no pierda la condición. En caso contrario, se debe bajar el dato más cercano de la página padre y fusionar las páginas que son hijas de éste.
    3.2. Si es menor se toma la dirección de sus descendientes por el lado izquierdo y se regresa al paso 1.
    3.3. Si es mayor, se avanza al siguiente dato de la misma página.
        3.3.1. Si es el último, se toma la dirección de sus descendientes por el lado derecho y se regresa al paso 1.
        3.3.2. Si no es el último, se regresa al paso 3.

El proceso de fusión puede llegar hasta la raíz, en cuyo caso la altura del árbol disminuye en uno. Cuando se llevan a cabo las fusiones todas aquellas claves copias de elementos eliminados en las páginas hojas



Árboles-B+

La tabla 7.13 presenta la secuencia de operaciones requeridas para llevar a cabo la eliminación del valor 261 del árbol-B+, de grado 2, de la figura 7.26.

1 Se recupera la página con el valor: 105.
2 Se evalúa si la página está vacía. No lo está.
3 Se compara el dato a eliminar (261) con el valor 105. Es mayor y no hay más elementos en la misma página, entonces se toma la dirección de la página que está a la derecha del 105.
4 Se recupera la página con los valores: 105 – 261.
5 Se compara el dato a eliminar (261) con el valor 105. Es mayor y hay más elementos en la misma página, entonces se avanza al siguiente valor (261).
6 Se compara el dato a eliminar (261) con el valor 261. Es igual.
7 Se evalúa si está en una página hoja. Sí lo está, entonces se elimina.
8 Se evalúa si el número de elementos que quedó en la página (1) es n y 2n. No lo es.
9 Se baja el dato más cercano de la página padre (105) y éste no se puede sustituir, ya que el número de elementos de su otro hijo es 2. Por lo tanto, se baja y se fusionan sus páginas descendientes. En este caso, la altura del árbol disminuye en uno. El proceso termina con éxito.

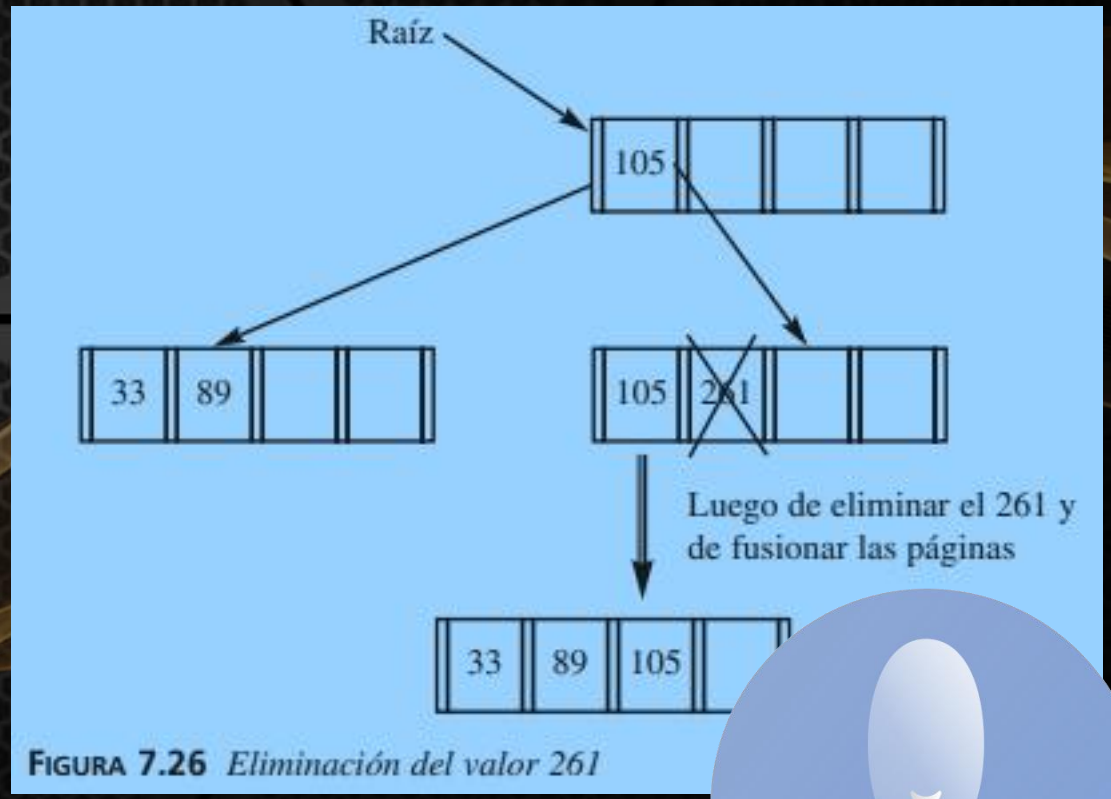


FIGURA 7.26 *Eliminación del valor 261*

TREES LABS

Laboratorio de Arboles



Muchas Gracias.

